

title: "AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT"

version: "V1.3"

status: "ARCHITECTURE FROZEN"

generated_at: "2026-08-16 03:45 UTC"

baseline: "V1.1"

revision_source: "V1.2"

AEGIS — AUTONOMOUS TRADING INTELLIGENCE SYSTEM

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

****ARCHITECTURE FROZEN****

>

This is the final V1.3 implementation authority.

>

V1.1 = historical architectural baseline.

>

V1.2 = revision/delta source.

>

V1.3 = consolidated authority.

Executive Summary

AEGIS V1.3 is an autonomous swing-trading platform using AI for market analysis and structured decision generation, deterministic risk control, broker execution, accounting, replay, paper trading, observability and auditability.

The architecture supports both:

SANDBOX

LIVE

Both are implemented through the same canonical pipeline.

SANDBOX is enabled by default.

LIVE is implemented but disabled by default.

Changing:

TRADING_ENVIRONMENT=SANDBOX

to:

TRADING_ENVIRONMENT=LIVE

must select the LiveBroker without a code change. Live execution still requires every mandatory LIVE safety gate to pass.

The V1 release target is:

V1_STATUS = PAPER_READY

This means the complete system is implemented and validated for replay/paper operation, while LIVE remains disabled by default and requires explicit operational authorization.

V1.3 — ARCHITECTURE AUTHORITY

1. Status

ARCHITECTURE FROZEN

This document is the sole implementation authority for V1.3.

The coding agent is an implementer, not the architect.

The auditor is a verifier, not an implementer.

No architectural decision may be silently introduced into code.

If a required architectural decision is missing:

STATUS = ARCHITECTURAL_BLOCKER

The agent must stop and request an architectural revision.

2. Scope

V1.3 is an autonomous swing-trading system for analysis, structured AI decisions, deterministic risk control, execution, accounting, replay, paper trading, observability, auditability, and controlled broker execution.

There is **no academic/thesis scope** in V1.3.

V1.3 supports two execution environments:

SANDBOX

LIVE

Both environments are implemented.

SANDBOX is the default and validation environment.

LIVE is fully implemented in the codebase but disabled by default and protected by mandatory safety gates.

3. Canonical Pipeline

Market Data

↓

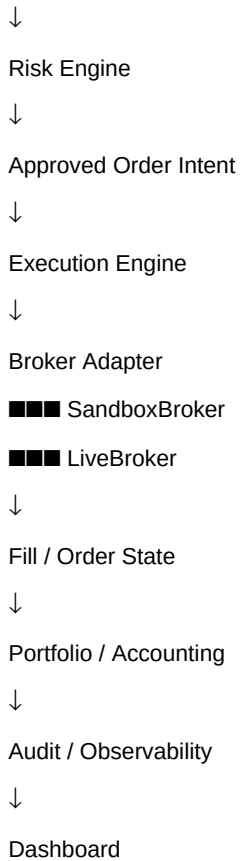
Context / Market State Builder

↓

AI Decision Engine

↓

Decision Contract



The same decision/risk/execution pipeline is used in SANDBOX and LIVE.

Only the broker adapter and environment-specific operational controls differ.

4. Environment Contract

TRADING_ENVIRONMENT = SANDBOX | LIVE

Default:

TRADING_ENVIRONMENT=SANDBOX

Required runtime controls:

LIVE_ENABLED=false

LIVE_CONFIRMATION_REQUIRED=true

Changing:

TRADING_ENVIRONMENT=SANDBOX

to:

TRADING_ENVIRONMENT=LIVE

must select LiveBroker without changing application code.

However, selecting LIVE does not itself authorize an order.

LIVE execution requires all mandatory gates to pass.

LIVE gate

TRADING_ENVIRONMENT == LIVE

↓

LIVE_ENABLED == true

↓

live confirmation valid

↓

live credentials valid

↓

broker connectivity valid

↓

permissions validated

↓

risk profile valid

↓

kill switch available and inactive

↓

live order/value/loss/position limits valid

↓

system health valid

↓

reconciliation valid

↓

EXECUTE

Any failed condition:

FAIL-CLOSED

↓

NO LIVE ORDER

Required properties

- LiveBroker is implemented.
- LiveBroker implements the same BrokerAdapter contract as SandboxBroker.
- LiveBroker cannot bypass Risk Engine.
- LiveBroker cannot be called directly by the LLM.
- LiveBroker cannot be called directly by the frontend.
- LIVE_ENABLED=false always blocks live execution.
- Invalid/missing live credentials block execution.
- Unknown order state requires reconciliation before continuation.
- Environment changes are auditable.

- Every live execution carries correlation_id and idempotency protection.

5. Environment Separation

Environment-specific secrets must be separate.

SANDBOX credentials must never be silently reused as LIVE credentials.

LIVE credentials must never be exposed to:

- frontend;
- LLM prompts;
- audit payloads;
- Git;
- source code;
- normal application logs.

The dashboard may display:

SANDBOX

LIVE (DISABLED)

LIVE (READY)

LIVE (BLOCKED)

but must never expose secret values.

6. Financial Safety

Critical monetary/quantity calculations use Decimal.

UTC is the internal time standard.

PostgreSQL is the financial source of truth.

Redis is not the financial source of truth.

No look-ahead is permitted.

Only information available at the decision timestamp may enter a decision.

Hard risk limits are deterministic and cannot be overridden by the LLM.

The LLM recommends.

The Risk Engine authorizes or rejects.

The Execution Engine executes only an approved Order Intent.

7. AI Authority

The LLM has no direct execution authority.

The LLM may:

- analyze market context;
- produce structured reasoning/decision output;
- provide confidence;
- recommend LONG/HOLD/CLOSE;
- provide requested trade parameters inside the Decision Contract.

The LLM may NOT:

- call a broker;
- submit an order;
- bypass Risk Engine;
- modify hard safety limits;
- access provider secrets;
- alter accounting;
- alter audit history.

Invalid or unavailable LLM output fails safely.

8. Replay

Replay is a first-class validation foundation, not a late feature.

Replay must reproduce historical decision points deterministically while respecting information availability at each timestamp.

Replay validates:

- market state;
- AI input/output;
- decisions;
- risk;
- order intents;
- state transitions;
- portfolio;
- audit reconstruction.

Replay must never authorize live execution.

9. Failure / Recovery

Startup:

Load Configuration

↓

Database Connectivity

↓

Load Portfolio

↓

Reconcile Orders

↓

Validate Positions

↓

Validate Risk

↓

Recover Workers

↓

RUN or PAUSE

If state is ambiguous:

PAUSE

Never guess.

LLM unavailable:

- no new entries;
- existing positions remain protected;
- deterministic risk/protection continues.

Market data inconsistent:

- no new decision.

Risk Engine unavailable:

- no execution.

Order state unknown:

- reconcile before continuing.

10. Security Invariants

Secrets:

- never committed to Git;
- never returned to frontend;
- never injected into LLM prompts;
- never stored in audit payloads;
- never hardcoded.

Frontend:

- cannot access secrets;
- cannot bypass Risk Engine;
- cannot directly create broker orders;
- cannot alter hard risk limits.

11. Observability

Track at minimum:

Technical:

- API latency;
- LLM latency;
- LLM token usage;
- worker queue depth;
- market data lag;
- WebSocket connections;
- DB latency;
- Redis latency;
- error rate.

AI:

- invalid output rate;
- timeout rate;
- risk rejection rate;
- LONG/HOLD/CLOSE distribution;
- confidence;
- decision consistency.

Trading:

- P&L;
- drawdown;
- win rate;
- profit factor;
- Sharpe;
- exposure;
- fees;
- slippage.

Every critical operation is auditable.

12. Dataset / Experiment Registry

Dataset:

dataset_id
source
version
symbols
timeframe
start
end
checksum
created_at

Experiment:

experiment_id
model
prompt_version
dataset_id
configuration
seed
results
created_at

V1.3 — FROZEN TECHNOLOGY STACK

| Layer | Technology |

|---|---|

| Runtime | Python 3.12+ |

| API | FastAPI |

| Validation | Pydantic v2 |

| ORM | SQLAlchemy 2.x |

| Migrations | Alembic |

| Database | PostgreSQL |

| Cache / messaging | Redis, only when necessary |

| Data / Quant | Polars / Pandas |

| Frontend | React + TypeScript + Vite |

| Testing | pytest |

| Containers | Docker + Docker Compose |

| LLM | Provider abstraction |

| Observability | Structured logs + metrics + audit trail |

Performance policy:

V1.3 is not optimized for low latency. The objective is determinism, security, testability, maintainability, and speed of development.

A future performance optimization requires objective evidence and an ADR.

V1.3 — ADR REGISTER

ADR-001 — AI Has No Direct Execution Authority

The LLM cannot call a broker or submit orders.

ADR-002 — Risk Engine Is Mandatory

Every Order Intent must pass deterministic Risk Engine validation.

ADR-003 — PostgreSQL Is Financial Source of Truth

Redis may support coordination/cache/messaging but cannot be the authoritative financial ledger.

ADR-004 — Decimal for Critical Financial Calculations

No binary floating-point arithmetic for critical monetary/quantity calculations.

ADR-005 — UTC

UTC is the internal time standard.

ADR-006 — Fail-Closed

Ambiguous, unavailable, or unsafe conditions block new execution.

ADR-007 — Replay Is First-Class

Replay is a foundational validation mechanism.

ADR-008 — Multi-Environment Execution

SANDBOX and LIVE implement the same BrokerAdapter contract and use the same decision/risk/execution pipeline.

ADR-009 — LIVE Implemented but Disabled by Default

LiveBroker is implemented in V1.3 but cannot execute while LIVE_ENABLED is false or any required live gate fails.

ADR-010 — Environment Selected by Configuration

TRADING_ENVIRONMENT=SANDBOX|LIVE selects the execution adapter without code changes. The selected environment is auditable.

ADR-011 — Architecture Freeze

Any unspecified architectural decision produces ARCHITECTURAL_BLOCKER.

ADR-012 — No Academic Scope

V1.3 has no thesis/academic deliverable requirement.

ADR Process

Proposal

↓

ADR

↓

Blueprint Version

↓

Implementation

No silent architectural decisions.

V1.3 — FORMAL DELTA / SUPERSESION MATRIX

The table below prevents the exact context-loss problem identified in the transition from V1.1 to V1.2.

ID	Earlier decision	V1.3 decision	Status
D-001	V1.1 baseline	Preserve all valid architectural content	PRESERVED
D-002	V1.2 Replay was late	Replay is first-class foundation	ADOPTED
D-003	V1.2 frozen stack	Freeze complete stack	ADOPTED
D-004	V1.2 measurable gates	Mandatory measurable gates	ADOPTED
D-005	V1.2 PASS_WITH_WARNINGS	Retained only when phase gate permits	ADOPTED
D-006	16 phases	Retain 16 architectural phases	CONFIRMED
D-007	Academic scope	Explicitly removed	SUPERSEDED
D-008	V1.2 repeated evidence blocks	Replace with this delta matrix	SUPERSEDED
D-009	V1.1/V1.2 LiveBroker absent	LiveBroker fully implemented in V1.3	SUPERSEDED
D-010	LIVE outside V1	LIVE implementation exists but is disabled	SUPERSEDED
D-011	Sandbox/Paper only	SANDBOX + LIVE environments	SUPERSEDED
D-012	Stack partially described	Full stack frozen	ADOPTED
D-013	Fase 16 generic release	Explicit final certification gate	ADOPTED
D-014	Coder could encounter ambiguity	ARCHITECTURAL_BLOCKER mandatory	PRESERVED
D-015	V1 final target PAPER_READY	PAPER_READY remains V1 release target	CONFIRMED

Important interpretation

"LIVE implemented" is not equivalent to "LIVE certified."

V1.3 implements the LiveBroker and all required code paths so that the environment can be switched without a code rewrite.

The V1 release certification remains:

V1_STATUS = PAPER_READY

LIVE execution requires its own explicit operational gates and is disabled by default.

This is deliberate separation between implementation completeness and operational authorization.

V1.3 — IMPLEMENTATION GOVERNANCE

Mandatory Phase Contract

Every phase MUST contain:

1. Objective
2. Scope
3. Out of Scope
4. Dependencies
5. Architectural View
6. Contracts Involved
7. Acceptance Criteria
8. Implementation Prompt
9. Audit Prompt
10. Gate
11. Required Evidence

The 16 phases are retained because they represent architectural boundaries and dependency boundaries, not artificial granularity.

Coding Agent Rules

Before coding:

- inspect repository;
- inspect existing implementation;
- read applicable V1.3 phase;
- read applicable ADRs;
- inspect dependencies from approved phases;
- do not reimplement existing contracts.

During coding:

- implement only authorized scope;
- write automated tests;
- preserve existing contracts;
- preserve state machines;
- preserve risk invariants;
- do not perform unrelated refactors;
- do not invent unspecified architecture.

At completion:

- run tests;

- list files changed;
- list tests executed;
- map acceptance criteria to evidence;
- report limitations;
- report ARCHITECTURAL_BLOCKERS;
- do not self-approve.

Auditor Rules

The auditor must inspect the actual repository and not trust claims.

The auditor must:

- execute relevant tests;
- verify acceptance criteria;
- inspect contracts;
- inspect state machines;
- search for out-of-scope functionality;
- search for bypasses;
- search for secrets;
- search for financial float usage;
- verify UTC;
- verify correlation_id;
- verify idempotency;
- verify fail-closed behavior;
- verify no look-ahead;
- verify LLM has no direct execution authority;
- verify frontend cannot bypass Risk;
- verify SandboxBroker and LiveBroker both implement BrokerAdapter;
- verify LIVE is blocked when LIVE_ENABLED=false;
- verify environment switching requires no code modification;
- verify live gates;
- verify reconciliation behavior;
- verify audit trail.

The auditor does not silently repair code.

Audit Status

PASS:

all mandatory criteria demonstrably satisfied.

PASS_WITH_WARNINGS:

only when the phase explicitly permits warnings and no mandatory safety or

architectural criterion is violated.

FAIL:

a required criterion is missing or violated.

ARCHITECTURAL_BLOCKER:

implementation requires a decision not specified by V1.3.

A phase cannot be approved merely because the project compiles.

Phase Approval Gate

APPROVED only when:

- implementation complete;
- unit tests pass;
- integration tests pass;
- phase-specific tests pass;
- architecture verified;
- security verified;
- audit executed;
- deviations corrected;
- evidence collected;
- Git commit recorded;
- PHASE_XX_APPROVED.md created;
- no ARCHITECTURAL_BLOCKER remains.

Required Evidence

Each phase must leave:

docs/phases/PHASE_XX_APPROVED.md

test output

relevant logs/evidence

Git commit

audit report

The approval file must identify:

- blueprint version;
- phase;
- commit;
- test results;
- audit status;
- acceptance evidence;
- auditor;

- timestamp.

V1.3 — 16-PHASE IMPLEMENTATION PLAN

FASE 01 — Foundation, Stack & Repository

Objetivo

Freeze stack, monorepo, Docker Compose, environment configuration, health checks, domain skeleton and CI-quality foundations.

Escopo

Freeze stack, monorepo, Docker Compose, environment configuration, health checks, domain skeleton and CI-quality foundations.

Fora do Escopo

No broker execution, no operational AI, no functional trading.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

→ Context

→ AI Decision

→ Decision Contract

→ Risk Engine

→ Approved Order Intent

→ Execution Engine

→ Broker Adapter

→ Fill

→ Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-01.01** — Python 3.12+ is used by the project.
- **AC-01.02** — FastAPI starts successfully.
- **AC-01.03** — Docker image builds successfully and reproducibly.
- **AC-01.04** — Docker Compose starts the services defined for the phase.
- **AC-01.05** — Configuration is loaded through environment/configuration mechanisms, not hardcoded.
- **AC-01.06** — TRADING_ENVIRONMENT accepts only SANDBOX or LIVE.
- **AC-01.07** — SANDBOX is the default trading environment.
- **AC-01.08** — LIVE_ENABLED defaults to false.
- **AC-01.09** — No secrets are hardcoded in source code.
- **AC-01.10** — Application health check responds successfully.
- **AC-01.11** — Automated test infrastructure is functional.
- **AC-01.12** — The project test command is reproducible.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:
AEGIS V1.3 – CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are NOT the architect.

Implement ONLY:

PHASE 01 – Foundation, Stack & Repository

SCOPE:
Freeze stack, monorepo, Docker Compose, environment configuration, health checks, domain skeleton and CI-quality foundations.

OUT OF SCOPE:
No broker execution, no operational AI, no functional trading.

ACCEPTANCE CRITERIA:

AC-01.01 | Python 3.12+ is used by the project.
AC-01.02 | FastAPI starts successfully.
AC-01.03 | Docker image builds successfully and reproducibly.
AC-01.04 | Docker Compose starts the services defined for the phase.
AC-01.05 | Configuration is loaded through environment/configuration mechanisms, not hardcoded.
AC-01.06 | TRADING_ENVIRONMENT accepts only SANDBOX or LIVE.
AC-01.07 | SANDBOX is the default trading environment.
AC-01.08 | LIVE_ENABLED defaults to false.
AC-01.09 | No secrets are hardcoded in source code.
AC-01.10 | Application health check responds successfully.
AC-01.11 | Automated test infrastructure is functional.
AC-01.12 | The project test command is reproducible.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.

11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
 STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.
LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.
Do NOT mark the phase approved.

FASE 02 – Domain Contracts & State Machines

Objetivo

Implement Domain Contracts, enums, Event Contracts, State Machines, Time Contract, correlation_id and idempotency conventions.

Escopo

Implement Domain Contracts, enums, Event Contracts, State Machines, Time Contract, correlation_id and idempotency conventions.

Fora do Escopo

No database persistence, broker calls or LLM execution.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data
→ Context
→ AI Decision
→ Decision Contract
→ Risk Engine
→ Approved Order Intent
→ Execution Engine
→ Broker Adapter
→ Fill
→ Portfolio
→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

```
TRADING_ENVIRONMENT=SANDBOX|LIVE
```

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-02.01** – Domain Contracts are explicitly defined.
- **AC-02.02** – Domain enums are centralized.
- **AC-02.03** – Order State Machine has explicit valid transitions.
- **AC-02.04** – Position State Machine has explicit valid transitions.
- **AC-02.05** – Decision State Machine has explicit valid transitions.
- **AC-02.06** – Invalid state transitions are rejected.
- **AC-02.07** – Domain events have validated schemas.
- **AC-02.08** – `correlation_id` is propagated through critical operations.
- **AC-02.09** – Idempotency keys are supported.
- **AC-02.10** – Internal timestamps use UTC.
- **AC-02.11** – Tests cover valid and invalid state transitions.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are NOT the architect.

Implement ONLY:

PHASE 02 — Domain Contracts & State Machines

SCOPE:

Implement Domain Contracts, enums, Event Contracts, State Machines, Time Contract, `correlation_id` and idempotency conventions.

OUT OF SCOPE:

No database persistence, broker calls or LLM execution.

ACCEPTANCE CRITERIA:

AC-02.01 | Domain Contracts are explicitly defined.

AC-02.02 | Domain enums are centralized.

AC-02.03 | Order State Machine has explicit valid transitions.

AC-02.04 | Position State Machine has explicit valid transitions.

AC-02.05 | Decision State Machine has explicit valid transitions.

AC-02.06 | Invalid state transitions are rejected.

AC-02.07 | Domain events have validated schemas.

AC-02.08 | `correlation_id` is propagated through critical operations.

AC-02.09 | Idempotency keys are supported.

AC-02.10 | Internal timestamps use UTC.

AC-02.11 | Tests cover valid and invalid state transitions.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.

LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;

- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 03 — PostgreSQL, ORM & Persistence

Objetivo

Implement PostgreSQL schema, SQLAlchemy models, Alembic migrations and repository patterns with PostgreSQL as financial source of truth.

Escopo

Implement PostgreSQL schema, SQLAlchemy models, Alembic migrations and repository patterns with PostgreSQL as financial source of truth.

Fora do Escopo

No live execution and no bypass of domain contracts.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

→ Context

→ AI Decision

→ Decision Contract

→ Risk Engine

→ Approved Order Intent

→ Execution Engine

→ Broker Adapter

→ Fill

→ Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-03.01** — PostgreSQL is the primary financial database.
- **AC-03.02** — SQLAlchemy 2.x is configured.
- **AC-03.03** — Alembic is configured.
- **AC-03.04** — Database migrations can be executed from an empty database.
- **AC-03.05** — Migration strategy is documented and reversible where applicable.
- **AC-03.06** — Required financial entities have persistence models.
- **AC-03.07** — Repository layer does not violate Domain Contracts.
- **AC-03.08** — Critical persistence operations have consistent transaction behavior.
- **AC-03.09** — Critical monetary and quantity values do not rely on binary floating point.
- **AC-03.10** — Persisted state survives application restart.
- **AC-03.11** — Persistence tests pass.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 – CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 03 – PostgreSQL, ORM & Persistence

SCOPE:

Implement PostgreSQL schema, SQLAlchemy models, Alembic migrations and repository patterns with PostgreSQL as financial source of truth.

OUT OF SCOPE:

No live execution and no bypass of domain contracts.

ACCEPTANCE CRITERIA:

AC-03.01 | PostgreSQL is the primary financial database.
AC-03.02 | SQLAlchemy 2.x is configured.
AC-03.03 | Alembic is configured.
AC-03.04 | Database migrations can be executed from an empty database.
AC-03.05 | Migration strategy is documented and reversible where applicable.
AC-03.06 | Required financial entities have persistence models.
AC-03.07 | Repository layer does not violate Domain Contracts.
AC-03.08 | Critical persistence operations have consistent transaction behavior.
AC-03.09 | Critical monetary and quantity values do not rely on binary floating point.
AC-03.10 | Persisted state survives application restart.
AC-03.11 | Persistence tests pass.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.

11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.
LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.
Do NOT mark the phase approved.

FASE 04 – Market Data & Context Builder

Objetivo

Implement market data ingestion, candle validation, closed-candle policy, normalization, Market State and Context Builder.

Escopo

Implement market data ingestion, candle validation, closed-candle policy, normalization, Market State and Context Builder.

Fora do Escopo

No trading decision execution.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

```
Market Data
→ Context
→ AI Decision
→ Decision Contract
→ Risk Engine
→ Approved Order Intent
→ Execution Engine
→ Broker Adapter
→ Fill
→ Portfolio
→ Audit.
```

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

```
TRADING_ENVIRONMENT=SANDBOX|LIVE
```

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-04.01** – Market data can be ingested through the defined abstraction.
- **AC-04.02** – Market timestamps are normalized to UTC.
- **AC-04.03** – Invalid market data is rejected or quarantined.
- **AC-04.04** – Candles pass consistency validation.
- **AC-04.05** – Closed-candle policy is explicitly enforced.
- **AC-04.06** – Market State is deterministic for the same input.
- **AC-04.07** – Context Builder never uses future information.
- **AC-04.08** – LLM context is reproducible from the recorded market state.
- **AC-04.09** – Market-data lag is detectable.
- **AC-04.10** – Tests demonstrate absence of look-ahead.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 04 — Market Data & Context Builder

SCOPE:

Implement market data ingestion, candle validation, closed-candle policy, normalization, Market State and Context Builder.

OUT OF SCOPE:

No trading decision execution.

ACCEPTANCE CRITERIA:

AC-04.01 | Market data can be ingested through the defined abstraction.

AC-04.02 | Market timestamps are normalized to UTC.

AC-04.03 | Invalid market data is rejected or quarantined.

AC-04.04 | Candles pass consistency validation.

AC-04.05 | Closed-candle policy is explicitly enforced.

AC-04.06 | Market State is deterministic for the same input.

AC-04.07 | Context Builder never uses future information.

AC-04.08 | LLM context is reproducible from the recorded market state.

AC-04.09 | Market-data lag is detectable.

AC-04.10 | Tests demonstrate absence of look-ahead.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.

LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 05 — LLM Provider & AI Decision Engine

Objetivo

Implement provider abstraction, model configuration, prompt versioning, AI runs, structured Decision Contract validation and safe failure.

Escopo

Implement provider abstraction, model configuration, prompt versioning, AI runs, structured Decision Contract validation and safe failure.

Fora do Escopo

LLM cannot call broker, Risk Engine or Execution Engine directly.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

→ Context

→ AI Decision

→ Decision Contract

→ Risk Engine

→ Approved Order Intent

→ Execution Engine

→ Broker Adapter

→ Fill

→ Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-05.01** — An abstract LLM provider interface exists.
- **AC-05.02** — Provider and model can be selected through configuration.
- **AC-05.03** — The selected provider/model is recorded for each AI run.
- **AC-05.04** — Prompt versions are explicit and traceable.
- **AC-05.05** — LLM inputs are recorded without secrets.
- **AC-05.06** — LLM output is validated against the Decision Contract.
- **AC-05.07** — Invalid LLM output is rejected safely.
- **AC-05.08** — LLM timeout produces safe behavior.
- **AC-05.09** — LLM has no direct broker access.
- **AC-05.10** — LLM cannot modify hard Risk limits.
- **AC-05.11** — Decision ID and correlation_id are preserved.
- **AC-05.12** — Tests cover valid, invalid and timeout responses.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 – CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 05 – LLM Provider & AI Decision Engine

SCOPE:

Implement provider abstraction, model configuration, prompt versioning, AI runs, structured Decision Contract validation and safe failure handling.

OUT OF SCOPE:

LLM cannot call broker, Risk Engine or Execution Engine directly.

ACCEPTANCE CRITERIA:

AC-05.01 | An abstract LLM provider interface exists.
AC-05.02 | Provider and model can be selected through configuration.
AC-05.03 | The selected provider/model is recorded for each AI run.
AC-05.04 | Prompt versions are explicit and traceable.
AC-05.05 | LLM inputs are recorded without secrets.
AC-05.06 | LLM output is validated against the Decision Contract.
AC-05.07 | Invalid LLM output is rejected safely.
AC-05.08 | LLM timeout produces safe behavior.
AC-05.09 | LLM has no direct broker access.
AC-05.10 | LLM cannot modify hard Risk limits.
AC-05.11 | Decision ID and correlation_id are preserved.
AC-05.12 | Tests cover valid, invalid and timeout responses.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.

8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.
LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.
Do NOT mark the phase approved.

FASE 06 – Deterministic Risk Engine

Objetivo

Implement deterministic risk limits, position sizing, exposure, stop requirements, daily loss limits, kill switch and reason codes.

Escopo

Implement deterministic risk limits, position sizing, exposure, stop requirements, daily loss limits, kill switch and reason codes.

Fora do Escopo

No risk bypass and no LLM override of hard limits.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

- Context
- AI Decision
- Decision Contract
- Risk Engine
- Approved Order Intent
- Execution Engine
- Broker Adapter
- Fill
- Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

```
TRADING_ENVIRONMENT=SANDBOX|LIVE
```

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-06.01** – Risk Engine accepts only valid Decision Contracts.
- **AC-06.02** – Position sizing is deterministic.
- **AC-06.03** – Exposure limits are enforced.
- **AC-06.04** – Maximum position size is enforced.
- **AC-06.05** – Required stop-loss conditions are validated.
- **AC-06.06** – Daily loss limit is enforced.
- **AC-06.07** – Kill switch blocks new orders.
- **AC-06.08** – Risk rejection produces a machine-readable reason code.
- **AC-06.09** – LLM cannot override hard Risk limits.
- **AC-06.10** – An unapproved order can never reach Execution Engine.
- **AC-06.11** – Critical financial calculations use Decimal.
- **AC-06.12** – Tests cover limits, boundary conditions and rejection paths.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are NOT the architect.

Implement ONLY:

PHASE 06 — Deterministic Risk Engine

SCOPE:

Implement deterministic risk limits, position sizing, exposure, stop requirements, daily loss limits, kill switch and reason codes.

OUT OF SCOPE:

No risk bypass and no LLM override of hard limits.

ACCEPTANCE CRITERIA:

AC-06.01 | Risk Engine accepts only valid Decision Contracts.

AC-06.02 | Position sizing is deterministic.

AC-06.03 | Exposure limits are enforced.

AC-06.04 | Maximum position size is enforced.

AC-06.05 | Required stop-loss conditions are validated.

AC-06.06 | Daily loss limit is enforced.

AC-06.07 | Kill switch blocks new orders.

AC-06.08 | Risk rejection produces a machine-readable reason code.

AC-06.09 | LLM cannot override hard Risk limits.

AC-06.10 | An unapproved order can never reach Execution Engine.

AC-06.11 | Critical financial calculations use Decimal.

AC-06.12 | Tests cover limits, boundary conditions and rejection paths.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:

STATUS = ARCHITECTURAL_BLOCKER

17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.

LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 07 — Portfolio & Accounting

Objetivo

Implement cash, positions, fills, realized/unrealized P&L, average cost, exposure and deterministic accounting.

Escopo

Implement cash, positions, fills, realized/unrealized P&L, average cost, exposure and deterministic accounting.

Fora do Escopo

Accounting must not depend on the LLM as source of truth.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

→ Context

→ AI Decision

→ Decision Contract

→ Risk Engine

→ Approved Order Intent

→ Execution Engine

→ Broker Adapter

→ Fill

→ Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-07.01** — Cash balance is maintained.
- **AC-07.02** — Positions are maintained.
- **AC-07.03** — Fills update positions correctly.
- **AC-07.04** — Average cost is calculated deterministically.
- **AC-07.05** — Realized P&L is calculated.
- **AC-07.06** — Unrealized P&L is calculated.
- **AC-07.07** — Fees are accounted for.
- **AC-07.08** — Exposure is calculated.
- **AC-07.09** — Accounting does not depend on LLM output.
- **AC-07.10** — Financial state survives restart.
- **AC-07.11** — Accounting tests cover entries, exits, partial fills and fees.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are NOT the architect.

Implement ONLY:

PHASE 07 — Portfolio & Accounting

SCOPE:

Implement cash, positions, fills, realized/unrealized P&L, average cost, exposure and deterministic accounting.

OUT OF SCOPE:

Accounting must not depend on the LLM as source of truth.

ACCEPTANCE CRITERIA:

AC-07.01 | Cash balance is maintained.
AC-07.02 | Positions are maintained.
AC-07.03 | Fills update positions correctly.
AC-07.04 | Average cost is calculated deterministically.
AC-07.05 | Realized P&L is calculated.
AC-07.06 | Unrealized P&L is calculated.
AC-07.07 | Fees are accounted for.
AC-07.08 | Exposure is calculated.
AC-07.09 | Accounting does not depend on LLM output.
AC-07.10 | Financial state survives restart.
AC-07.11 | Accounting tests cover entries, exits, partial fills and fees.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.

14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.
LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 08 – Broker Contract & Sandbox Execution

Objetivo

Implement BrokerAdapter contract, SandboxBroker, order lifecycle, idempotency and simulated fills.

Escopo

Implement BrokerAdapter contract, SandboxBroker, order lifecycle, idempotency and simulated fills.

Fora do Escopo

Broker execution must never bypass Risk Engine.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

```
Market Data
→ Context
→ AI Decision
→ Decision Contract
→ Risk Engine
→ Approved Order Intent
→ Execution Engine
→ Broker Adapter
→ Fill
→ Portfolio
→ Audit.
```

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-08.01** – BrokerAdapter contract is explicitly defined.
- **AC-08.02** – SandboxBroker implements BrokerAdapter.
- **AC-08.03** – Order submission works in Sandbox.
- **AC-08.04** – Order cancellation works in Sandbox.
- **AC-08.05** – Order status retrieval works.
- **AC-08.06** – Sandbox fill simulation works.
- **AC-08.07** – Idempotency prevents duplicate orders.
- **AC-08.08** – Order lifecycle follows the Order State Machine.
- **AC-08.09** – Broker cannot receive an order without Risk approval.
- **AC-08.10** – Sandbox never uses LIVE credentials.
- **AC-08.11** – Sandbox execution tests pass.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are NOT the architect.

Implement ONLY:

PHASE 08 — Broker Contract & Sandbox Execution

SCOPE:

Implement BrokerAdapter contract, SandboxBroker, order lifecycle, idempotency and simulated fills.

OUT OF SCOPE:

Broker execution must never bypass Risk Engine.

ACCEPTANCE CRITERIA:

AC-08.01 | BrokerAdapter contract is explicitly defined.

AC-08.02 | SandboxBroker implements BrokerAdapter.

AC-08.03 | Order submission works in Sandbox.

AC-08.04 | Order cancellation works in Sandbox.

AC-08.05 | Order status retrieval works.

AC-08.06 | Sandbox fill simulation works.

AC-08.07 | Idempotency prevents duplicate orders.

AC-08.08 | Order lifecycle follows the Order State Machine.

AC-08.09 | Broker cannot receive an order without Risk approval.

AC-08.10 | Sandbox never uses LIVE credentials.

AC-08.11 | Sandbox execution tests pass.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:

STATUS = ARCHITECTURAL_BLOCKER

17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.

LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 09 — Live Broker Implementation & Environment Switching

Objetivo

Implement LiveBroker using the same BrokerAdapter contract, environment selection and mandatory LIVE safety gates.

Escopo

Implement LiveBroker using the same BrokerAdapter contract, environment selection and mandatory LIVE safety gates.

Fora do Escopo

LIVE must remain disabled by default and cannot bypass safety gates.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

→ Context

→ AI Decision

→ Decision Contract

→ Risk Engine

→ Approved Order Intent

→ Execution Engine

→ Broker Adapter

→ Fill

→ Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-09.01** — LiveBroker is implemented.
- **AC-09.02** — LiveBroker fully implements BrokerAdapter.
- **AC-09.03** — Sandbox and Live use the same execution contract.
- **AC-09.04** — TRADING_ENVIRONMENT=SANDBOX selects SandboxBroker.
- **AC-09.05** — TRADING_ENVIRONMENT=LIVE selects LiveBroker.
- **AC-09.06** — Changing environment requires no application-code modification.
- **AC-09.07** — LIVE_ENABLED=false blocks every LIVE order attempt.
- **AC-09.08** — Invalid or missing LIVE credentials block execution.
- **AC-09.09** — Broker connectivity failure blocks LIVE execution.
- **AC-09.10** — Failed broker health check blocks LIVE execution.
- **AC-09.11** — Invalid LIVE risk configuration blocks execution.
- **AC-09.12** — Active kill switch blocks LIVE execution.
- **AC-09.13** — LIVE order, exposure, loss and position limits are enforced.
- **AC-09.14** — LiveBroker cannot be called directly by the LLM.
- **AC-09.15** — LiveBroker cannot be called directly by the frontend.
- **AC-09.16** — LiveBroker cannot bypass Risk Engine.
- **AC-09.17** — Every LIVE execution attempt is auditable.
- **AC-09.18** — LIVE secrets never appear in logs, frontend or audit payloads.
- **AC-09.19** — Fail-closed behavior is covered by automated tests.
- **AC-09.20** — SANDBOX→LIVE switching is proven without code changes.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:
AEGIS V1.3 – CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 09 – Live Broker Implementation & Environment Switching

SCOPE:

Implement LiveBroker using the same BrokerAdapter contract, environment selection and mandatory LIVE safety gates.

OUT OF SCOPE:

LIVE must remain disabled by default and cannot bypass safety gates.

ACCEPTANCE CRITERIA:

AC-09.01 | LiveBroker is implemented.
AC-09.02 | LiveBroker fully implements BrokerAdapter.
AC-09.03 | Sandbox and Live use the same execution contract.
AC-09.04 | TRADING_ENVIRONMENT=SANDBOX selects SandboxBroker.
AC-09.05 | TRADING_ENVIRONMENT=LIVE selects LiveBroker.
AC-09.06 | Changing environment requires no application-code modification.
AC-09.07 | LIVE_ENABLED=false blocks every LIVE order attempt.
AC-09.08 | Invalid or missing LIVE credentials block execution.

AC-09.09 | Broker connectivity failure blocks LIVE execution.
 AC-09.10 | Failed broker health check blocks LIVE execution.
 AC-09.11 | Invalid LIVE risk configuration blocks execution.
 AC-09.12 | Active kill switch blocks LIVE execution.
 AC-09.13 | LIVE order, exposure, loss and position limits are enforced.
 AC-09.14 | LiveBroker cannot be called directly by the LLM.
 AC-09.15 | LiveBroker cannot be called directly by the frontend.
 AC-09.16 | LiveBroker cannot bypass Risk Engine.
 AC-09.17 | Every LIVE execution attempt is auditable.
 AC-09.18 | LIVE secrets never appear in logs, frontend or audit payloads.
 AC-09.19 | Fail-closed behavior is covered by automated tests.
 AC-09.20 | SANDBOX→LIVE switching is proven without code changes.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
 STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.
 LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.
 Do NOT mark the phase approved.

FASE 10 – Execution Engine, Reconciliation & Recovery

Objetivo

Implement orchestration from approved Order Intent to broker adapter, ACK/fill tracking, reconciliation, restart recovery and fail-closed b

Escopo

Implement orchestration from approved Order Intent to broker adapter, ACK/fill tracking, reconciliation, restart recovery and fail-closed b

```

## Fora do Escopo

No execution from unapproved intents.

## Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data
→ Context
→ AI Decision
→ Decision Contract
→ Risk Engine
→ Approved Order Intent
→ Execution Engine
→ Broker Adapter
→ Fill
→ Portfolio
→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

    TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

---

## Acceptance Criteria

The following criteria are the authoritative acceptance contract for this
phase.

The coding agent MUST implement them.

The auditor MUST verify every criterion individually.

- AC-10.01 – Only Approved Order Intent can be executed.
- AC-10.02 – Execution Engine uses BrokerAdapter.
- AC-10.03 – Order acknowledgement is processed.
- AC-10.04 – Partial fills are processed.
- AC-10.05 – Full fills are processed.
- AC-10.06 – Cancellation is processed.
- AC-10.07 – Unknown order state triggers reconciliation.
- AC-10.08 – Application restart triggers required reconciliation.
- AC-10.09 – System never assumes order success without confirmation.
- AC-10.10 – Idempotency prevents duplicate order submission.
- AC-10.11 – Risk Engine failure blocks execution.
- AC-10.12 – Recovery behavior is tested.

---

```

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 10 — Execution Engine, Reconciliation & Recovery

SCOPE:

Implement orchestration from approved Order Intent to broker adapter, ACK/fill tracking, reconciliation, restart recovery and fail-closed behavior.

OUT OF SCOPE:

No execution from unapproved intents.

ACCEPTANCE CRITERIA:

AC-10.01 | Only Approved Order Intent can be executed.

AC-10.02 | Execution Engine uses BrokerAdapter.

AC-10.03 | Order acknowledgement is processed.

AC-10.04 | Partial fills are processed.

AC-10.05 | Full fills are processed.

AC-10.06 | Cancellation is processed.

AC-10.07 | Unknown order state triggers reconciliation.

AC-10.08 | Application restart triggers required reconciliation.

AC-10.09 | System never assumes order success without confirmation.

AC-10.10 | Idempotency prevents duplicate order submission.

AC-10.11 | Risk Engine failure blocks execution.

AC-10.12 | Recovery behavior is tested.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:

STATUS = ARCHITECTURAL_BLOCKER

17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.

LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 11 — Replay Engine

Objetivo

Implement deterministic historical replay with timestamp-correct information availability and no look-ahead.

Escopo

Implement deterministic historical replay with timestamp-correct information availability and no look-ahead.

Fora do Escopo

Replay cannot authorize or send LIVE execution.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

→ Context

→ AI Decision

→ Decision Contract

→ Risk Engine

→ Approved Order Intent

→ Execution Engine

→ Broker Adapter

→ Fill

→ Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-11.01** — Replay accepts a versioned dataset.
- **AC-11.02** — Replay preserves historical timestamps.
- **AC-11.03** — Replay uses only information available at each timestamp.
- **AC-11.04** — Look-ahead is impossible or explicitly detected.
- **AC-11.05** — Historical Market State can be reconstructed.
- **AC-11.06** — AI Decision can be reproduced or deterministically stubbed.
- **AC-11.07** — Risk decisions can be reproduced.
- **AC-11.08** — Order Intents can be reproduced.
- **AC-11.09** — Portfolio state can be reconstructed.
- **AC-11.10** — Replay audit trail is reconstructible.
- **AC-11.11** — Replay cannot invoke LIVE execution.
- **AC-11.12** — Repeated replay with identical inputs is deterministic within defined tolerances.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:
AEGIS V1.3 – CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 11 – Replay Engine

SCOPE:
Implement deterministic historical replay with timestamp-correct information availability and no look-ahead.

OUT OF SCOPE:
Replay cannot authorize or send LIVE execution.

ACCEPTANCE CRITERIA:

AC-11.01 | Replay accepts a versioned dataset.
AC-11.02 | Replay preserves historical timestamps.
AC-11.03 | Replay uses only information available at each timestamp.
AC-11.04 | Look-ahead is impossible or explicitly detected.
AC-11.05 | Historical Market State can be reconstructed.
AC-11.06 | AI Decision can be reproduced or deterministically stubbed.
AC-11.07 | Risk decisions can be reproduced.
AC-11.08 | Order Intents can be reproduced.
AC-11.09 | Portfolio state can be reconstructed.
AC-11.10 | Replay audit trail is reconstructible.
AC-11.11 | Replay cannot invoke LIVE execution.
AC-11.12 | Repeated replay with identical inputs is deterministic within defined tolerances.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.

11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.
LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.
Do NOT mark the phase approved.

FASE 12 – Backtest, Metrics & Experiment Registry

Objetivo

Implement backtest execution, dataset registry, experiment registry, metrics, reproducibility and result persistence.

Escopo

Implement backtest execution, dataset registry, experiment registry, metrics, reproducibility and result persistence.

Fora do Escopo

Backtest cannot execute real broker orders.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

```
Market Data
→ Context
→ AI Decision
→ Decision Contract
→ Risk Engine
→ Approved Order Intent
→ Execution Engine
→ Broker Adapter
→ Fill
→ Portfolio
→ Audit.
```

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-12.01** – Each dataset has an ID and version.
- **AC-12.02** – Each dataset has a checksum.
- **AC-12.03** – Each experiment has an ID.
- **AC-12.04** – Model identity is recorded.
- **AC-12.05** – Prompt version is recorded.
- **AC-12.06** – Experiment configuration is recorded.
- **AC-12.07** – Seed is recorded when applicable.
- **AC-12.08** – Backtest results are persisted.
- **AC-12.09** – P&L is calculated.
- **AC-12.10** – Drawdown is calculated.
- **AC-12.11** – Win rate is calculated.
- **AC-12.12** – Profit factor is calculated.
- **AC-12.13** – Sharpe is calculated when applicable.
- **AC-12.14** – Backtest cannot submit real broker orders.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 12 — Backtest, Metrics & Experiment Registry

SCOPE:

Implement backtest execution, dataset registry, experiment registry, metrics, reproducibility and result persistence.

OUT OF SCOPE:

Backtest cannot execute real broker orders.

ACCEPTANCE CRITERIA:

AC-12.01 | Each dataset has an ID and version.

AC-12.02 | Each dataset has a checksum.

AC-12.03 | Each experiment has an ID.

AC-12.04 | Model identity is recorded.

AC-12.05 | Prompt version is recorded.

AC-12.06 | Experiment configuration is recorded.

AC-12.07 | Seed is recorded when applicable.

AC-12.08 | Backtest results are persisted.

AC-12.09 | P&L is calculated.

AC-12.10 | Drawdown is calculated.

AC-12.11 | Win rate is calculated.

AC-12.12 | Profit factor is calculated.

AC-12.13 | Sharpe is calculated when applicable.

AC-12.14 | Backtest cannot submit real broker orders.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:

STATUS = ARCHITECTURAL_BLOCKER

17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.

LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 13 — Audit, Observability & Security

Objetivo

Implement end-to-end audit trail, structured logs, metrics, secret handling, security controls and trace reconstruction.

Escopo

Implement end-to-end audit trail, structured logs, metrics, secret handling, security controls and trace reconstruction.

Fora do Escopo

No secret leakage.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

→ Context

→ AI Decision

→ Decision Contract

→ Risk Engine

→ Approved Order Intent

→ Execution Engine

→ Broker Adapter

→ Fill

→ Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-13.01** — Every critical operation produces an audit event.
- **AC-13.02** — `correlation_id` enables end-to-end operation tracing.
- **AC-13.03** — Audit records decision, risk, order and outcome.
- **AC-13.04** — Logs are structured.
- **AC-13.05** — Required operational metrics are available.
- **AC-13.06** — LLM latency is observable.
- **AC-13.07** — Market-data lag is observable.
- **AC-13.08** — Risk rejection is observable.
- **AC-13.09** — Secrets never appear in logs.
- **AC-13.10** — Secrets never appear in audit records.
- **AC-13.11** — Secrets never reach the frontend.
- **AC-13.12** — Security scan finds no hardcoded secrets.
- **AC-13.13** — A trading operation can be reconstructed from the audit trail.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:
AEGIS V1.3 – CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are NOT the architect.

Implement ONLY:

PHASE 13 – Audit, Observability & Security

SCOPE:
Implement end-to-end audit trail, structured logs, metrics, secret handling, security controls and trace reconstruction.

OUT OF SCOPE:
No secret leakage.

ACCEPTANCE CRITERIA:

AC-13.01 | Every critical operation produces an audit event.
AC-13.02 | `correlation_id` enables end-to-end operation tracing.
AC-13.03 | Audit records decision, risk, order and outcome.
AC-13.04 | Logs are structured.
AC-13.05 | Required operational metrics are available.
AC-13.06 | LLM latency is observable.
AC-13.07 | Market-data lag is observable.
AC-13.08 | Risk rejection is observable.
AC-13.09 | Secrets never appear in logs.
AC-13.10 | Secrets never appear in audit records.
AC-13.11 | Secrets never reach the frontend.
AC-13.12 | Security scan finds no hardcoded secrets.
AC-13.13 | A trading operation can be reconstructed from the audit trail.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.

8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.
LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.
Do NOT mark the phase approved.

FASE 14 – Dashboard & Operational UI

Objetivo

Implement dashboard for positions, P&L, orders, audit, risk, environment, provider/LLM configuration and operational status.

Escopo

Implement dashboard for positions, P&L, orders, audit, risk, environment, provider/LLM configuration and operational status.

Fora do Escopo

Frontend cannot execute broker orders or alter hard risk rules.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

- Context
- AI Decision
- Decision Contract
- Risk Engine
- Approved Order Intent
- Execution Engine
- Broker Adapter
- Fill
- Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

```
TRADING_ENVIRONMENT=SANDBOX|LIVE
```

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-14.01** – Dashboard displays current trading environment.
- **AC-14.02** – Dashboard displays system health/status.
- **AC-14.03** – Dashboard displays open positions.
- **AC-14.04** – Dashboard displays P&L.
- **AC-14.05** – Dashboard displays orders.
- **AC-14.06** – Dashboard displays exposure.
- **AC-14.07** – Dashboard displays Risk status.
- **AC-14.08** – Provider/LLM configuration is visible without exposing secrets.
- **AC-14.09** – Configured stop-loss settings can be viewed/managed according to authorized permissions.
- **AC-14.10** – Dashboard clearly displays LIVE state as DISABLED, BLOCKED or READY.
- **AC-14.11** – Frontend cannot call Broker directly.
- **AC-14.12** – Frontend cannot bypass Risk Engine.
- **AC-14.13** – Frontend cannot alter hard safety limits.
- **AC-14.14** – Critical actions require confirmation and produce audit events.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 14 — Dashboard & Operational UI

SCOPE:

Implement dashboard for positions, P&L, orders, audit, risk, environment, provider/LLM configuration and operational status.

OUT OF SCOPE:

Frontend cannot execute broker orders or alter hard risk rules.

ACCEPTANCE CRITERIA:

AC-14.01 | Dashboard displays current trading environment.

AC-14.02 | Dashboard displays system health/status.

AC-14.03 | Dashboard displays open positions.

AC-14.04 | Dashboard displays P&L.

AC-14.05 | Dashboard displays orders.

AC-14.06 | Dashboard displays exposure.

AC-14.07 | Dashboard displays Risk status.

AC-14.08 | Provider/LLM configuration is visible without exposing secrets.

AC-14.09 | Configured stop-loss settings can be viewed/managed according to authorized permissions.

AC-14.10 | Dashboard clearly displays LIVE state as DISABLED, BLOCKED or READY.

AC-14.11 | Frontend cannot call Broker directly.

AC-14.12 | Frontend cannot bypass Risk Engine.

AC-14.13 | Frontend cannot alter hard safety limits.

AC-14.14 | Critical actions require confirmation and produce audit events.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:

STATUS = ARCHITECTURAL_BLOCKER

17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.

LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 15 — E2E, Chaos, Recovery & Release Gates

Objetivo

Implement comprehensive E2E, failure injection, recovery, backup/restore, environment-switch and LIVE safety-gate tests.

Escopo

Implement comprehensive E2E, failure injection, recovery, backup/restore, environment-switch and LIVE safety-gate tests.

Fora do Escopo

Do not certify unrestricted production LIVE operation.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

Market Data

→ Context

→ AI Decision

→ Decision Contract

→ Risk Engine

→ Approved Order Intent

→ Execution Engine

→ Broker Adapter

→ Fill

→ Portfolio

→ Audit.

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

TRADING_ENVIRONMENT=SANDBOX|LIVE

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-15.01** — Complete SANDBOX pipeline operates successfully.
- **AC-15.02** — Market Data→AI→Risk→Execution→Fill→Portfolio→Audit works end-to-end.
- **AC-15.03** — LLM timeout is handled safely.
- **AC-15.04** — Market-data failure is handled safely.
- **AC-15.05** — Database failure is handled safely.
- **AC-15.06** — Redis failure is handled safely when Redis is used.
- **AC-15.07** — Broker failure is handled safely.
- **AC-15.08** — Unknown order state is reconciled.
- **AC-15.09** — Restart recovery works.
- **AC-15.10** — Backup and restore are verified.
- **AC-15.11** — Kill switch works end-to-end.
- **AC-15.12** — Risk limits work end-to-end.
- **AC-15.13** — Environment switching works without code modification.
- **AC-15.14** — LIVE remains blocked when LIVE_ENABLED=false.
- **AC-15.15** — All mandatory LIVE safety gates are tested.
- **AC-15.16** — Replay works end-to-end.
- **AC-15.17** — Backtest works end-to-end.
- **AC-15.18** — Audit reconstruction works end-to-end.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:
AEGIS V1.3 – CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 15 – E2E, Chaos, Recovery & Release Gates

SCOPE:
Implement comprehensive E2E, failure injection, recovery, backup/restore, environment-switch and LIVE safety-gate tests.

OUT OF SCOPE:
Do not certify unrestricted production LIVE operation.

ACCEPTANCE CRITERIA:

AC-15.01 | Complete SANDBOX pipeline operates successfully.
AC-15.02 | Market Data→AI→Risk→Execution→Fill→Portfolio→Audit works end-to-end.
AC-15.03 | LLM timeout is handled safely.
AC-15.04 | Market-data failure is handled safely.
AC-15.05 | Database failure is handled safely.
AC-15.06 | Redis failure is handled safely when Redis is used.
AC-15.07 | Broker failure is handled safely.
AC-15.08 | Unknown order state is reconciled.
AC-15.09 | Restart recovery works.
AC-15.10 | Backup and restore are verified.
AC-15.11 | Kill switch works end-to-end.
AC-15.12 | Risk limits work end-to-end.

AC-15.13 | Environment switching works without code modification.
 AC-15.14 | LIVE remains blocked when LIVE_ENABLED=false.
 AC-15.15 | All mandatory LIVE safety gates are tested.
 AC-15.16 | Replay works end-to-end.
 AC-15.17 | Backtest works end-to-end.
 AC-15.18 | Audit reconstruction works end-to-end.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.
13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
 STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.
 LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

FASE 16 – Final Certification

Objetivo

Execute final certification sequence, verify every phase, invariant, release gate and produce final certification artifacts.

Escopo

Execute final certification sequence, verify every phase, invariant, release gate and produce final certification artifacts.

Fora do Escopo

Do not introduce new architecture during certification.

Visão Arquitetural

Implement strictly within the V1.3 architecture.

Preserve existing Domain Contracts and State Machines.

The canonical pipeline is:

```
Market Data
→ Context
→ AI Decision
→ Decision Contract
→ Risk Engine
→ Approved Order Intent
→ Execution Engine
→ Broker Adapter
→ Fill
→ Portfolio
→ Audit.
```

SANDBOX and LIVE share the same decision/risk/execution pipeline.

Environment selection is configuration-driven:

```
TRADING_ENVIRONMENT=SANDBOX|LIVE
```

SANDBOX is the default.

LIVE is fully implemented but disabled by default.

LIVE_ENABLED=false must fail-closed.

Acceptance Criteria

The following criteria are the **authoritative acceptance contract** for this phase.

The coding agent **MUST** implement them.

The auditor **MUST** verify every criterion individually.

- **AC-16.01** – All previous 15 phases have formal approval.
- **AC-16.02** – No ARCHITECTURAL_BLOCKER remains open.
- **AC-16.03** – No mandatory FAIL remains open.
- **AC-16.04** – Complete automated test suite passes.
- **AC-16.05** – End-to-end suite passes.
- **AC-16.06** – Replay certification passes.
- **AC-16.07** – Backtest certification passes.
- **AC-16.08** – Security audit passes.
- **AC-16.09** – Chaos and recovery certification passes.
- **AC-16.10** – Backup/restore certification passes.
- **AC-16.11** – Sandbox execution certification passes.
- **AC-16.12** – LiveBroker implementation is certified against BrokerAdapter.
- **AC-16.13** – LIVE safety gates are certified.
- **AC-16.14** – LIVE_ENABLED=false is proven to prevent LIVE execution.
- **AC-16.15** – Environment switching is proven without code modification.
- **AC-16.16** – Audit trail integrity and reconstruction are proven.
- **AC-16.17** – No secret leakage is identified.
- **AC-16.18** – No Risk Engine bypass is identified.
- **AC-16.19** – No LLM→Broker execution path exists.
- **AC-16.20** – No Frontend→Broker execution path exists.
- **AC-16.21** – V1_STATUS is PAPER_READY.
- **AC-16.22** – LIVE remains IMPLEMENTED + DISABLED BY DEFAULT.

Prompt de Implementação

You are the coding agent for AEGIS V1.3.

AUTHORITATIVE DOCUMENT:

AEGIS V1.3 — CONSOLIDATED & ARCHITECTURE-FROZEN MASTER BLUEPRINT

You are **NOT** the architect.

Implement **ONLY**:

PHASE 16 — Final Certification

SCOPE:

Execute final certification sequence, verify every phase, invariant, release gate and produce final certification artifacts.

OUT OF SCOPE:

Do not introduce new architecture during certification.

ACCEPTANCE CRITERIA:

- AC-16.01 | All previous 15 phases have formal approval.
- AC-16.02 | No ARCHITECTURAL_BLOCKER remains open.
- AC-16.03 | No mandatory FAIL remains open.
- AC-16.04 | Complete automated test suite passes.
- AC-16.05 | End-to-end suite passes.
- AC-16.06 | Replay certification passes.
- AC-16.07 | Backtest certification passes.
- AC-16.08 | Security audit passes.
- AC-16.09 | Chaos and recovery certification passes.
- AC-16.10 | Backup/restore certification passes.
- AC-16.11 | Sandbox execution certification passes.
- AC-16.12 | LiveBroker implementation is certified against BrokerAdapter.
- AC-16.13 | LIVE safety gates are certified.
- AC-16.14 | LIVE_ENABLED=false is proven to prevent LIVE execution.
- AC-16.15 | Environment switching is proven without code modification.
- AC-16.16 | Audit trail integrity and reconstruction are proven.
- AC-16.17 | No secret leakage is identified.
- AC-16.18 | No Risk Engine bypass is identified.
- AC-16.19 | No LLM→Broker execution path exists.
- AC-16.20 | No Frontend→Broker execution path exists.
- AC-16.21 | V1_STATUS is PAPER_READY.
- AC-16.22 | LIVE remains IMPLEMENTED + DISABLED BY DEFAULT.

MANDATORY ARCHITECTURAL RULES:

1. Do not invent architecture.
2. Do not change Domain Contracts without a formal V1.3 revision.
3. Do not change State Machines without a formal V1.3 revision.
4. Do not bypass Risk Engine.
5. Do not give the LLM execution authority.
6. Do not let Frontend call Broker directly.
7. Use Decimal for critical financial calculations.
8. Use UTC internally.
9. Preserve correlation_id.
10. Preserve idempotency.
11. Never introduce look-ahead.
12. Never hardcode secrets.

13. Never expose secrets to frontend, prompts or audit payloads.
14. PostgreSQL remains financial source of truth.
15. Fail closed on ambiguous or unsafe conditions.
16. If architecture is unspecified or contradictory, STOP and report:
STATUS = ARCHITECTURAL_BLOCKER
17. Do not silently resolve architectural contradictions.
18. Write automated tests for implemented behavior.
19. Do not modify unrelated components.
20. Do not declare the phase APPROVED.

ENVIRONMENT RULE:

SANDBOX is the default.

LIVE is implemented but disabled by default.

If this phase touches execution:

TRADING_ENVIRONMENT=SANDBOX|LIVE

must select the appropriate broker adapter without application-code changes.

LIVE_ENABLED=false

must always fail-closed.

IMPLEMENTATION REQUIREMENT:

For EVERY Acceptance Criterion:

- implement the required behavior;
- create or update an automated test where applicable;
- preserve evidence showing how the criterion was satisfied.

At completion report:

- files created/changed;
- tests executed;
- Acceptance Criteria status;
- evidence location for each AC;
- architectural decisions encountered;
- ARCHITECTURAL_BLOCKERS, if any;
- limitations;
- proposed commit message.

Do NOT claim PASS.

Do NOT mark the phase approved.

V1.3 — FINAL CERTIFICATION

Phase certification

ALL 16 PHASES PASS

↓

FULL E2E

↓

REPLAY

↓

BACKTEST

↓

SECURITY AUDIT

↓

CHAOS / RESILIENCE

↓

RECOVERY

↓

BACKUP / RESTORE

↓

PAPER TRADING

↓

ENVIRONMENT SWITCH TEST

↓

LIVE SAFETY-GATE TESTS

↓

AUDIT TRAIL INTEGRITY

↓

FINAL APPROVAL

↓

V1_STATUS = PAPER_READY

Live implementation certification

The final V1 certification must also prove:

- LiveBroker is implemented.
- LiveBroker conforms to BrokerAdapter.
- LiveBroker cannot bypass Risk Engine.
- LIVE_ENABLED=false blocks live orders.
- missing credentials block live orders.
- failed health checks block live orders.

- failed reconciliation blocks live orders.
- invalid risk configuration blocks live orders.
- kill switch blocks live orders.
- environment selection changes adapter without code changes.
- live orders preserve idempotency and correlation_id.
- live order state is reconciled.
- live execution is auditable.

Important

The V1 system is therefore:

SANDBOX = IMPLEMENTED + DEFAULT + ENABLED

LIVE = IMPLEMENTED + DISABLED BY DEFAULT

This does NOT mean LIVE is certified for unrestricted production use.

Operational LIVE authorization is a separate release decision.

Final Invariants

Never permit:

LLM → Broker

Frontend → Broker

Risk bypass

Unknown-order continuation

Ambiguous-state continuation

Secret exposure

Look-ahead

Financial float for critical calculations

LIVE execution when any gate fails

APPENDIX A — HISTORICAL V1.1 BASELINE

The complete V1.1 source is preserved below for traceability.

IMPORTANT:

V1.3 is the current authority. Any clause in this historical appendix that conflicts with the V1.3 Supersession Matrix is superseded by V1.3.

This preserves context without allowing obsolete V1.1 decisions to control implementation.

BLUEPRINT V1.1 — AUTONOMOUS SWING TRADING BOT

Status: FINAL / ARCHITECTURE FROZEN

Data de geração: 2026-08-15

Documento mestre de arquitetura e implementação da V1.

Ambiente exclusivamente Sandbox / Paper / Demo.

A V1 não possui Live Broker.

1. Identidade do Projeto

- ****name:**** Autonomous Swing Trading Bot
- ****version:**** V1.1
- ****status:**** FINAL / ARCHITECTURE FROZEN
- ****environment:**** Sandbox / Paper / Demo
- ****reference_capital:**** R\$ 100,00
- ****objective:**** Projeto educacional e experimental
- ****implementation_phases:**** 16
- ****live_broker:**** NÃO IMPLEMENTADO NA V1

2. Objetivo

Construir um sistema autônomo de swing trading orientado por IA, executado exclusivamente em ambiente experimental/sandbox/paper, com separação rígida entre análise da IA, gerenciamento de risco, execução simulada, contabilidade, observabilidade e interface.

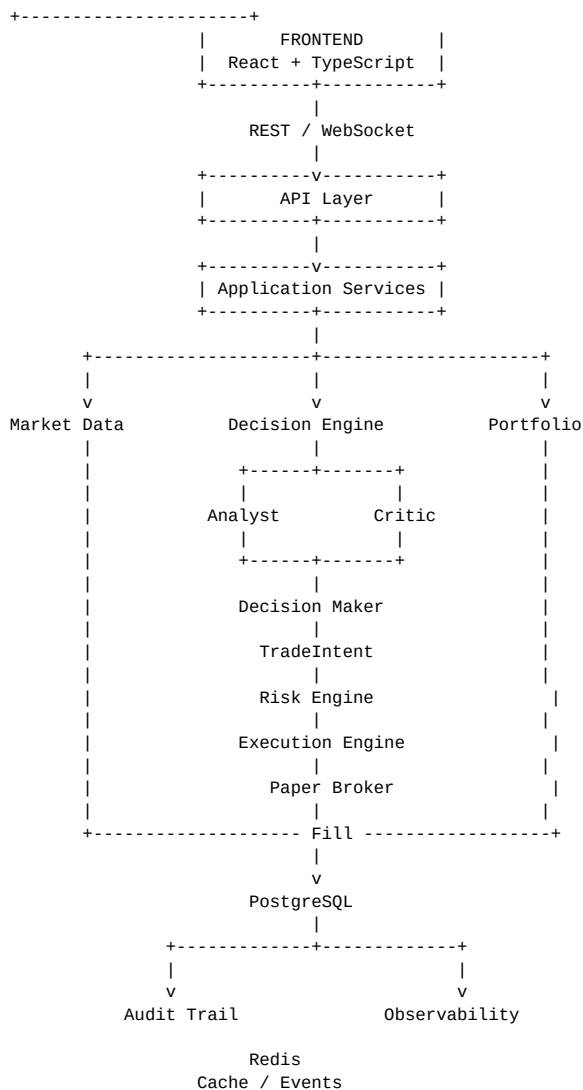
O sistema deve ser auditável, reproduzível, resiliente e incapaz de executar operações reais na V1.

3. Princípios Arquiteturais

- LLM nunca possui autoridade direta de execução.
- Risk Engine é determinístico.
- V1 não possui Live Broker.
- PostgreSQL é a fonte de verdade.
- Redis é cache/coordenação e não fonte de verdade financeira.
- Sistema opera em fail-closed.
- Stops e proteção de posições não dependem da disponibilidade do LLM.
- UTC é o timezone interno.
- Cálculos financeiros críticos utilizam Decimal.
- Nenhuma decisão pode utilizar informação futura.
- Toda operação crítica deve ser idempotente.

- Toda operação importante deve possuir `correlation_id`.
- Hard limits não podem ser alterados pela IA ou pela UI.
- Mudanças arquiteturais exigem ADR e nova versão do Blueprint.
- Requisito arquitetural não especificado deve gerar `ARCHITECTURAL_BLOCKER`.

4. Arquitetura



5. Domain Contracts

MarketState

```

market_state_id
asset
timestamp
timeframe
ohlcv
indicators
market_context
data_quality
source
hash

```

TradeIntent

```

trade_intent_id

```

asset
action
quantity
entry_price
stop_loss
take_profit
confidence
thesis
invalidation
created_at
market_state_id
ai_run_id

RiskDecision

risk_decision_id
trade_intent_id
status
approved_quantity
approved_price
risk_amount
exposure
reasons
created_at

OrderRequest

execution_id
trade_intent_id
risk_decision_id
client_order_id
asset
side
type
quantity
price
stop_loss
take_profit
created_at
idempotency_key

Order

order_id
client_order_id
status
filled_quantity
remaining_quantity
average_price
fees
created_at
updated_at

Fill

fill_id
order_id
quantity
price
fee
timestamp

Position

position_id
asset
side
quantity
average_entry
current_price
stop_loss
take_profit
realized_pnl
unrealized_pnl
opened_at
updated_at

PortfolioSnapshot

```
snapshot_id
timestamp
cash
equity
exposure
realized_pnl
unrealized_pnl
drawdown
```

AIRun

```
ai_run_id
agent
provider
model
prompt_version
input_hash
output_hash
started_at
completed_at
status
latency_ms
token_usage
```

AuditEvent

```
audit_event_id
correlation_id
event_type
entity_type
entity_id
timestamp
actor
payload_hash
```

6. State Machines

Order

```
CREATED
|
v
SUBMITTED
|
v
ACKNOWLEDGED
|
v
PARTIALLY_FILLED
|
v
FILLED
```

Terminal states:

```
CANCELLED
REJECTED
EXPIRED
ERROR
```

Terminal states cannot transition back to active states.

Position

```
NONE
|
v
OPENING
|
v
OPEN
|
v
CLOSING
|
```

v
CLOSED

AI Run

CREATED
 |
 v
RUNNING
 |
 v
COMPLETED

Alternative terminal states:

 FAILED
 TIMEOUT
 REJECTED

System

RUNNING
 |
 +----> PAUSED
 |
 +----> EMERGENCY_STOP

PAUSED:
 no new entries.

EMERGENCY_STOP:
 no new execution.

AI cannot modify system safety state.

7. Event Model

Every important operation must carry:

 correlation_id
 event_time
 ingestion_time
 processing_time

When applicable:

 execution_time

Typical event flow:

MarketDataReceived
 ->
CandleClosed
 ->
MarketStateCreated
 ->
AIRunStarted
 ->
AIRunCompleted
 ->
TradeIntentCreated
 ->
RiskDecisionCreated
 ->
OrderCreated
 ->
OrderSubmitted
 ->
OrderAcknowledged
 ->
FillReceived
 ->
PositionUpdated
 ->
PortfolioSnapshotCreated
 ->
AuditEventRecorded

No component may consume information whose event_time is later than the MarketState timestamp used for a decision.

8. Risk Model

Reference capital:
R\$ 100,00

Maximum risk per trade:
1%

Maximum simultaneous positions:
1

Mandatory stop:
YES

Circuit breaker:
10% drawdown

V1 execution:
PAPER / DEMO ONLY

Hard safety limits cannot be overridden by:

- LLM
- frontend
- user configuration
- provider response

9. AI Architecture

```
MarketState
|
v
Analyst
|
v
Critic
|
v
Decision Maker
|
v
TradeIntent
|
v
Risk Engine
```

The LLM may:

- analyze market information
- produce structured reasoning
- suggest LONG / SHORT / HOLD / CLOSE
- suggest entry/SL/TP
- provide confidence
- provide thesis/invalidation

The LLM may NOT:

- call the broker
- submit orders
- change hard risk limits
- change kill switch
- access secrets
- bypass validation
- modify accounting

10. Configuration Hierarchy

Authority hierarchy:

1. HARD SAFETY LIMITS
2. APPLICATION CONFIGURATION
3. USER CONFIGURATION
4. AI SUGGESTION

Lower levels cannot override higher levels.

Examples of user-configurable settings:

- LLM provider
- model
- prompt version
- risk parameters within hard limits
- stop parameters within allowed range
- market universe

- fees/slippage for simulation

Examples of hard limits:

- maximum risk
- maximum position count
- emergency stop
- live broker disabled

11. Failure & Recovery

Startup:

Application Start

->

Load Configuration

->

Database Connectivity

->

Load Portfolio

->

Reconcile Orders

->

Validate Positions

->

Validate Risk

->

Recover Workers

->

RUN or PAUSE

If state is ambiguous:

PAUSE

Never guess.

LLM unavailable:

- no new entries
- existing positions remain protected
- deterministic risk/protection continues

Market data inconsistent:

- no new decision

Risk Engine unavailable:

- no execution

Order state unknown:

- reconcile before continuing

12. Observability

Technical metrics:

API latency
LLM latency
LLM token usage
worker queue depth
market data lag
WebSocket connections
DB latency
Redis latency
error rate

AI metrics:

invalid output rate
timeout rate
risk rejection rate
LONG/HOLD/CLOSE distribution
average confidence
decision consistency

Trading metrics:

P&L
drawdown
win rate
profit factor
Sharpe
exposure
fees
slippage

13. Dataset & Experiment Registry

Dataset:

```
dataset_id
source
version
symbols
timeframe
start
end
checksum
created_at
```

Experiment:

```
experiment_id
model
prompt_version
dataset_id
configuration
seed
results
created_at
```

14. Security Model

Secrets:

- never committed to Git
- never returned to frontend
- never injected into LLM prompts
- never stored in audit payloads

V1:

- no live exchange credentials
- no LiveBroker implementation
- no production execution endpoint

API:

- authentication boundary
- authorization boundary
- input validation
- rate limiting where applicable

Frontend:

- cannot access provider secrets
- cannot bypass Risk Engine
- cannot directly create broker orders

15. ADR Governance

Directory:

```
docs/
  adr/
    ADR-001-ai-no-execution.md
    ADR-002-paper-only.md
    ADR-003-postgres-source-of-truth.md
    ADR-004-deterministic-risk.md
    ADR-005-utc.md
    ADR-006-fail-closed.md
```

Process:

Proposal

->

ADR

->

Blueprint version

->

Implementation

No architectural decision may be silently introduced into code.

16. Definition of Done

A phase is complete only when:

- [] implementation complete
- [] unit tests passing
- [] integration tests passing
- [] phase-specific tests passing
- [] architecture verified
- [] security verified
- [] audit prompt executed
- [] deviations corrected
- [] evidence collected
- [] Git commit recorded
- [] PHASE_XX_APPROVED.md created

Final V1:

V1_STATUS = PAPER_READY

NOT:

V1_STATUS = LIVE_READY

17. IMPLEMENTATION PLAN V1.1

A V1.1 utiliza 16 fases. O número não é arbitrário: as fases foram organizadas por fronteiras arquiteturais e dependências reais.

Cada fase contém:

- objetivo;
- escopo;
- fora do escopo;
- dependências;
- visão arquitetural;
- critérios de aceite;
- prompt de implementação;
- prompt de auditoria.

FASE 01 — Foundation & Architecture Contracts

Objetivo

Criar o esqueleto do projeto e materializar no código os contratos arquiteturais que serão utilizados por todas as demais fases.

Escopo

Docker/Compose, estrutura do projeto, configuração base, Domain Contracts, enums, State Machines, Event Contracts, Time Contract, UTC, correlation_id, idempotency conventions, estrutura de ADR e Architecture Freeze.

Fora do Escopo

Banco de dados, trading, broker, IA operacional, frontend funcional e execução.

Dependências

Nenhuma. É a fase inicial.

Visão Arquitetural

Esta fase estabelece a linguagem formal do sistema.

Os contratos devem ser independentes de infraestrutura sempre que possível.

O objetivo é impedir que fases posteriores inventem entidades, estados ou formatos incompatíveis.

Critérios de Aceite

- Projeto sobe através do mecanismo de containerização definido.
- Domain Contracts existem e possuem validação.
- Enums oficiais estão centralizados.
- State Machines estão formalizadas e testadas.
- UTC é a convenção interna.
- Correlation ID e idempotency conventions estão definidas.
- ADR structure existe.
- ARCHITECTURAL_BLOCKER está definido como mecanismo de governança.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 01 – FOUNDATION & ARCHITECTURE CONTRACTS

=====

OBJETIVO:

Criar o esqueleto do projeto e materializar no código os contratos arquiteturais que serão utilizados por todas as demais fases.

ESCOPO:

Docker/Compose, estrutura do projeto, configuração base,

Domain Contracts, enums, State Machines, Event Contracts, Time Contract, UTC, correlation_id, idempotency conventions, estrutura de ADR e Architecture Freeze.

FORA DO ESCOPO:

Banco de dados, trading, broker, IA operacional, frontend funcional e execução.

DEPENDÊNCIAS:

Nenhuma. É a fase inicial.

VISÃO ARQUITETURAL:

Esta fase estabelece a linguagem formal do sistema. Os contratos devem ser independentes de infraestrutura sempre que possível.

O objetivo é impedir que fases posteriores inventem entidades, estados ou formatos incompatíveis.

CRITÉRIOS DE ACEITE:

- Projeto sobre através do mecanismo de containerização definido.
- Domain Contracts existem e possuem validação.
- Enums oficiais estão centralizados.
- State Machines estão formalizadas e testadas.
- UTC é a convenção interna.
- Correlation ID e idempotency conventions estão definidas.
- ADR structure existe.
- ARCHITECTURAL_BLOCKER está definido como mecanismo de governança.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão. A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.

9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 01 – FOUNDATION & ARCHITECTURE CONTRACTS

=====

OBJETIVO ESPERADO:

Criar o esqueleto do projeto e materializar no código os contratos arquiteturais que serão utilizados por todas as demais fases.

ESCOPO ESPERADO:

Docker/Compose, estrutura do projeto, configuração base, Domain Contracts, enums, State Machines, Event Contracts, Time Contract, UTC, correlation_id, idempotency conventions, estrutura de ADR e Architecture Freeze.

FORA DO ESCOPO:

Banco de dados, trading, broker, IA operacional, frontend funcional e execução.

DEPENDÊNCIAS:

Nenhuma. É a fase inicial.

VISÃO ARQUITETURAL:

Esta fase estabelece a linguagem formal do sistema.
Os contratos devem ser independentes de infraestrutura
sempre que possível.

O objetivo é impedir que fases posteriores inventem
entidades, estados ou formatos incompatíveis.

CRITÉRIOS DE ACEITE:

- Projeto sobe através do mecanismo de containerização definido.
- Domain Contracts existem e possuem validação.
- Enums oficiais estão centralizados.
- State Machines estão formalizadas e testadas.
- UTC é a convenção interna.
- Correlation ID e idempotency conventions estão definidas.
- ADR structure existe.
- ARCHITECTURAL_BLOCKER está definido como mecanismo de governança.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 02 — Infrastructure & Persistence

Objetivo

Implementar persistência e infraestrutura base sem criar uma segunda fonte de verdade.

Escopo

PostgreSQL, Redis, migrations, repositories, persistência dos contratos, configuração de infraestrutura, backup e restore.

Fora do Escopo

Estratégia, IA, execução real, dashboard completo.

Dependências

Fase 01.

Visão Arquitetural

PostgreSQL é a fonte de verdade para estado financeiro, ordens, fills, posições, decisões e auditoria.

Redis é utilizado apenas para cache, coordenação e dados efêmeros.

Critérios de Aceite

- PostgreSQL inicializa corretamente.
- Migrations são reproduzíveis.
- Repositories possuem testes.
- Redis funciona sem ser usado como source of truth.
- Backup é produzido.
- Restore foi efetivamente testado.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 02 – INFRASTRUCTURE & PERSISTENCE

=====

OBJETIVO:

Implementar persistência e infraestrutura base sem criar uma segunda fonte de verdade.

ESCOPO:

PostgreSQL, Redis, migrations, repositories, persistência dos contratos, configuração de infraestrutura, backup e restore.

FORA DO ESCOPO:

Estratégia, IA, execução real, dashboard completo.

DEPENDÊNCIAS:

Fase 01.

VISÃO ARQUITETURAL:

PostgreSQL é a fonte de verdade para estado financeiro, ordens, fills, posições, decisões e auditoria.

Redis é utilizado apenas para cache, coordenação e dados efêmeros.

CRITÉRIOS DE ACEITE:

- PostgreSQL inicializa corretamente.
- Migrations são reproduzíveis.
- Repositories possuem testes.
- Redis funciona sem ser usado como source of truth.
- Backup é produzido.
- Restore foi efetivamente testado.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 02 – INFRASTRUCTURE & PERSISTENCE

=====

OBJETIVO ESPERADO:

Implementar persistência e infraestrutura base sem criar uma segunda fonte de verdade.

ESCOPO ESPERADO:

PostgreSQL, Redis, migrations, repositories, persistência dos contratos, configuração de infraestrutura, backup e restore.

FORA DO ESCOPO:

Estratégia, IA, execução real, dashboard completo.

DEPENDÊNCIAS:

Fase 01.

VISÃO ARQUITETURAL:

PostgreSQL é a fonte de verdade para estado financeiro, ordens, fills, posições, decisões e auditoria.

Redis é utilizado apenas para cache, coordenação e dados efêmeros.

CRITÉRIOS DE ACEITE:

- PostgreSQL inicializa corretamente.
- Migrations são reproduzíveis.
- Repositories possuem testes.
- Redis funciona sem ser usado como source of truth.
- Backup é produzido.
- Restore foi efetivamente testado.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.

11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 03 — Backend Core & Application Runtime

Objetivo

Criar o runtime principal da aplicação.

Escopo

API, application services, dependency injection, structured logging, error handling, health checks, configuration, workers, scheduler e estados RUNNING/PAUSED/EMERGENCY_STOP.

Fora do Escopo

Decision Engine, LLM operacional, Paper Broker.

Dependências

Fases 01 e 02.

Visão Arquitetural

O backend deve possuir separação clara entre domínio, aplicação e infraestrutura.

O runtime precisa suportar restart e recuperação sem reconstruir estado por inferência.

Critérios de Aceite

- Backend inicia e responde health checks.
- Workers possuem ciclo de vida controlado.
- Structured logging está ativo.
- RUNNING/PAUSED/EMERGENCY_STOP funcionam.
- Restart não corrompe estado persistido.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====
FASE 03 – BACKEND CORE & APPLICATION RUNTIME
=====

OBJETIVO:

Criar o runtime principal da aplicação.

ESCOPO:

API, application services, dependency injection, structured logging, error handling, health checks, configuration, workers, scheduler e estados RUNNING/PAUSED/EMERGENCY_STOP.

FORA DO ESCOPO:

Decision Engine, LLM operacional, Paper Broker.

DEPENDÊNCIAS:

Fases 01 e 02.

VISÃO ARQUITETURAL:

O backend deve possuir separação clara entre domínio, aplicação e infraestrutura.

O runtime precisa suportar restart e recuperação sem reconstruir estado por inferência.

CRITÉRIOS DE ACEITE:

- Backend inicia e responde health checks.
- Workers possuem ciclo de vida controlado.
- Structured logging está ativo.
- RUNNING/PAUSED/EMERGENCY_STOP funcionam.
- Restart não corrompe estado persistido.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:
PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:
resumo objetivo

IMPLEMENTED:
o que foi encontrado implementado

MISSING:
o que deveria existir e não existe

OUT_OF_SCOPE:
funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:
violações encontradas

TEST_RESULTS:
testes executados e resultado

SECURITY_RESULTS:
resultado

EVIDENCE:
arquivos, testes e evidências

RECOMMENDATION:
APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 03 – BACKEND CORE & APPLICATION RUNTIME

=====

OBJETIVO ESPERADO:

Criar o runtime principal da aplicação.

ESCOPO ESPERADO:

API, application services, dependency injection, structured logging, error handling, health checks, configuration, workers, scheduler e estados RUNNING/PAUSED/EMERGENCY_STOP.

FORA DO ESCOPO:

Decision Engine, LLM operacional, Paper Broker.

DEPENDÊNCIAS:

Fases 01 e 02.

VISÃO ARQUITETURAL:

O backend deve possuir separação clara entre domínio, aplicação e infraestrutura.

O runtime precisa suportar restart e recuperação sem reconstruir estado por inferência.

CRITÉRIOS DE ACEITE:

- Backend inicia e responde health checks.
- Workers possuem ciclo de vida controlado.
- Structured logging está ativo.
- RUNNING/PAUSED/EMERGENCY_STOP funcionam.
- Restart não corrompe estado persistido.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 04 — Market Data Engine

Objetivo

Implementar ingestão, normalização e persistência dos dados de mercado.

Escopo

Provider Adapter, historical data, candles, normalization, closed-candle detection, gaps, deduplicação, cache e persistência.

Fora do Escopo

Decisão de trading e execução.

Dependências

Fases 01-03.

Visão Arquitetural

O Market Data Engine deve produzir dados temporais auditáveis e impedir look-ahead.

Apenas candles CLOSED podem alimentar decisões.

Critérios de Aceite

- Candles são normalizados.
- Eventos duplicados são tratados.
- Gaps são detectados.
- Candle aberto não alimenta decisão.
- Timestamp é tratado em UTC.
- Não existe look-ahead.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 04 – MARKET DATA ENGINE

=====

OBJETIVO:

Implementar ingestão, normalização e persistência dos dados de mercado.

ESCOPO:

Provider Adapter, historical data, candles, normalization, closed-candle detection, gaps, deduplicação, cache e persistência.

FORA DO ESCOPO:

Decisão de trading e execução.

DEPENDÊNCIAS:

Fases 01-03.

VISÃO ARQUITETURAL:

O Market Data Engine deve produzir dados temporais auditáveis e impedir look-ahead.

Apenas candles CLOSED podem alimentar decisões.

CRITÉRIOS DE ACEITE:

- Candles são normalizados.
- Eventos duplicados são tratados.
- Gaps são detectados.
- Candle aberto não alimenta decisão.
- Timestamp é tratado em UTC.
- Não existe look-ahead.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 04 – MARKET DATA ENGINE

=====

OBJETIVO ESPERADO:

Implementar ingestão, normalização e persistência dos dados de mercado.

ESCOPO ESPERADO:

Provider Adapter, historical data, candles, normalization, closed-candle detection, gaps, deduplicação, cache e persistência.

FORA DO ESCOPO:

Decisão de trading e execução.

DEPENDÊNCIAS:

Fases 01-03.

VISÃO ARQUITETURAL:

O Market Data Engine deve produzir dados temporais auditáveis e impedir look-ahead.

Apenas candles CLOSED podem alimentar decisões.

CRITÉRIOS DE ACEITE:

- Candles são normalizados.
- Eventos duplicados são tratados.
- Gaps são detectados.
- Candle aberto não alimenta decisão.
- Timestamp é tratado em UTC.
- Não existe look-ahead.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.

11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 05 — Universe & Market State

Objetivo

Criar a seleção de ativos e o MarketState formal.

Escopo

Universe Scanner, critérios de elegibilidade, liquidez, volume, spread, disponibilidade, qualidade dos dados e construção do MarketState.

Fora do Escopo

Ordem, broker, IA de decisão.

Dependências

Fase 04.

Visão Arquitetural

Universe define quais ativos podem ser analisados.
MarketState representa somente informações disponíveis naquele instante.

Critérios de Aceite

- Universe é determinístico.
- MarketState possui hash.
- MarketState referencia fonte e timestamp.
- Dados futuros são rejeitados.
- Ativos sem dados suficientes não entram no pipeline.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

```
=====
FASE 05 – UNIVERSE & MARKET STATE
=====
```

OBJETIVO:

Criar a seleção de ativos e o MarketState formal.

ESCOPO:

Universe Scanner, critérios de elegibilidade, liquidez, volume, spread, disponibilidade, qualidade dos dados e construção do MarketState.

FORA DO ESCOPO:

Ordem, broker, IA de decisão.

DEPENDÊNCIAS:

Fase 04.

VISÃO ARQUITETURAL:

Universe define quais ativos podem ser analisados.
MarketState representa somente informações disponíveis naquele instante.

CRITÉRIOS DE ACEITE:

- Universe é determinístico.
- MarketState possui hash.
- MarketState referencia fonte e timestamp.

- Dados futuros são rejeitados.
- Ativos sem dados suficientes não entram no pipeline.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 05 – UNIVERSE & MARKET STATE

=====

OBJETIVO ESPERADO:

Criar a seleção de ativos e o MarketState formal.

ESCOPO ESPERADO:

Universe Scanner, critérios de elegibilidade, liquidez, volume, spread, disponibilidade, qualidade dos dados e construção do MarketState.

FORA DO ESCOPO:

Ordem, broker, IA de decisão.

DEPENDÊNCIAS:

Fase 04.

VISÃO ARQUITETURAL:

Universe define quais ativos podem ser analisados.
MarketState representa somente informações disponíveis naquele instante.

CRITÉRIOS DE ACEITE:

- Universe é determinístico.
- MarketState possui hash.
- MarketState referencia fonte e timestamp.
- Dados futuros são rejeitados.
- Ativos sem dados suficientes não entram no pipeline.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 06 — Paper Broker & Execution

Objetivo

Implementar a execução simulada e a máquina de estados de ordens.

Escopo

TradeIntent validation, Execution Engine, Paper Broker, MARKET/LIMIT, fills, partial fills, fees, spread, slippage, cancelamento, rejeição, retries e idempotência.

Fora do Escopo

Live Broker.

Dependências

Fases 01-05.

Visão Arquitetural

TradeIntent não é Order.

O fluxo obrigatório é:

TradeIntent

->

Execution Validation

->

Execution Engine

->

Paper Broker

->

Order/Fill

Critérios de Aceite

- Order State Machine é respeitada.
- Duplicate request não gera segunda ordem.
- Retry não duplica execução.
- Partial fill é tratado.

- Fees e slippage são registrados.
- Nenhum LiveBroker existe.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 06 – PAPER BROKER & EXECUTION

=====

OBJETIVO:

Implementar a execução simulada e a máquina de estados de ordens.

ESCOPO:

TradeIntent validation, Execution Engine, Paper Broker, MARKET/LIMIT, fills, partial fills, fees, spread, slippage, cancelamento, rejeição, retries e idempotência.

FORA DO ESCOPO:

Live Broker.

DEPENDÊNCIAS:

Fases 01-05.

VISÃO ARQUITETURAL:

TradeIntent não é Order.

O fluxo obrigatório é:

```
TradeIntent
->
Execution Validation
->
Execution Engine
->
Paper Broker
->
Order/Fill
```

CRITÉRIOS DE ACEITE:

- Order State Machine é respeitada.
- Duplicate request não gera segunda ordem.
- Retry não duplica execução.
- Partial fill é tratado.
- Fees e slippage são registrados.
- Nenhum LiveBroker existe.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.

3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 06 – PAPER BROKER & EXECUTION

=====

OBJETIVO ESPERADO:

Implementar a execução simulada e a máquina de estados de ordens.

ESCOPO ESPERADO:

TradeIntent validation, Execution Engine, Paper Broker, MARKET/LIMIT, fills, partial fills, fees, spread, slippage, cancelamento, rejeição, retries e idempotência.

FORA DO ESCOPO:

Live Broker.

DEPENDÊNCIAS:

Fases 01-05.

VISÃO ARQUITETURAL:

TradeIntent não é Order.

O fluxo obrigatório é:

```
TradeIntent
->
Execution Validation
->
Execution Engine
->
Paper Broker
->
Order/Fill
```

CRITÉRIOS DE ACEITE:

- Order State Machine é respeitada.
- Duplicate request não gera segunda ordem.
- Retry não duplica execução.
- Partial fill é tratado.
- Fees e slippage são registrados.
- Nenhum LiveBroker existe.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 07 — Portfolio & Accounting

Objetivo

Implementar contabilidade determinística do portfólio.

Escopo

Cash, equity, positions, average entry, realized P&L, unrealized P&L, fees, exposure e drawdown.

Fora do Escopo

IA, estratégia e dashboard completo.

Dependências

Fase 06.

Visão Arquitetural

Fill é a fonte do evento de alteração de posição.

Accounting utiliza Decimal para cálculos financeiros críticos.

Critérios de Aceite

- Cálculos utilizam Decimal.
- Compra funciona.
- Venda funciona.
- Partial fill funciona.
- Fees são contabilizados.
- P&L realizado funciona.
- P&L não realizado funciona.
- Posição zerada é tratada corretamente.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 07 — PORTFOLIO & ACCOUNTING

=====

OBJETIVO:

Implementar contabilidade determinística do portfólio.

ESCOPO:

Cash, equity, positions, average entry, realized P&L, unrealized P&L, fees, exposure e drawdown.

FORA DO ESCOPO:

IA, estratégia e dashboard completo.

DEPENDÊNCIAS:

Fase 06.

VISÃO ARQUITETURAL:

Fill é a fonte do evento de alteração de posição.
Accounting utiliza Decimal para cálculos financeiros críticos.

CRITÉRIOS DE ACEITE:

- Cálculos utilizam Decimal.
- Compra funciona.
- Venda funciona.

- Partial fill funciona.
- Fees são contabilizados.
- P&L realizado funciona.
- P&L não realizado funciona.
- Posição zerada é tratada corretamente.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 07 – PORTFOLIO & ACCOUNTING

=====

OBJETIVO ESPERADO:

Implementar contabilidade determinística do portfólio.

ESCOPO ESPERADO:

Cash, equity, positions, average entry, realized P&L,
unrealized P&L, fees, exposure e drawdown.

FORA DO ESCOPO:

IA, estratégia e dashboard completo.

DEPENDÊNCIAS:

Fase 06.

VISÃO ARQUITETURAL:

Fill é a fonte do evento de alteração de posição.
Accounting utiliza Decimal para cálculos financeiros críticos.

CRITÉRIOS DE ACEITE:

- Cálculos utilizam Decimal.
- Compra funciona.
- Venda funciona.
- Partial fill funciona.
- Fees são contabilizados.
- P&L realizado funciona.
- P&L não realizado funciona.
- Posição zerada é tratada corretamente.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturas.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 08 — Risk Engine & Safety

Objetivo

Criar a barreira determinística de risco.

Escopo

Position sizing, max risk, max positions, mandatory stop, exposure, circuit breaker, capital validation, tick size, step size, min notional, kill switch, hard limits e fail-closed.

Fora do Escopo

LLM, estratégia e execução direta.

Dependências

Fases 01, 06 e 07.

Visão Arquitetural

TradeIntent entra no Risk Engine e sai como RiskDecision.

O Risk Engine pode rejeitar qualquer decisão.

IA e frontend não podem ultrapassar seus limites.

Critérios de Aceite

- Risco máximo por operação: 1%.
- Máximo de posições: 1.
- Stop obrigatório.
- Circuit breaker de 10% configurado.
- Hard limits não podem ser sobrescritos.
- Risk Engine indisponível implica NO EXECUTION.
- Testes de bypass falham de forma segura.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 08 – RISK ENGINE & SAFETY

=====

OBJETIVO:

Criar a barreira determinística de risco.

ESCOPO:

Position sizing, max risk, max positions, mandatory stop, exposure, circuit breaker, capital validation, tick size, step size, min notional, kill switch, hard limits e fail-closed.

FORA DO ESCOPO:

LLM, estratégia e execução direta.

DEPENDÊNCIAS:

Fases 01, 06 e 07.

VISÃO ARQUITETURAL:

TradeIntent entra no Risk Engine e sai como RiskDecision.

O Risk Engine pode rejeitar qualquer decisão.

IA e frontend não podem ultrapassar seus limites.

CRITÉRIOS DE ACEITE:

- Risco máximo por operação: 1%.
- Máximo de posições: 1.
- Stop obrigatório.
- Circuit breaker de 10% configurado.
- Hard limits não podem ser sobrescritos.
- Risk Engine indisponível implica NO EXECUTION.
- Testes de bypass falham de forma segura.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 08 – RISK ENGINE & SAFETY

=====

OBJETIVO ESPERADO:

Criar a barreira determinística de risco.

ESCOPO ESPERADO:

Position sizing, max risk, max positions, mandatory stop,
exposure, circuit breaker, capital validation, tick size,
step size, min notional, kill switch, hard limits e fail-closed.

FORA DO ESCOPO:

LLM, estratégia e execução direta.

DEPENDÊNCIAS:

Fases 01, 06 e 07.

VISÃO ARQUITETURAL:

TradeIntent entra no Risk Engine e sai como RiskDecision.

O Risk Engine pode rejeitar qualquer decisão.

IA e frontend não podem ultrapassar seus limites.

CRITÉRIOS DE ACEITE:

- Risco máximo por operação: 1%.
- Máximo de posições: 1.
- Stop obrigatório.
- Circuit breaker de 10% configurado.
- Hard limits não podem ser sobrescritos.
- Risk Engine indisponível implica NO EXECUTION.
- Testes de bypass falham de forma segura.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.

10. Verifique `correlation_id` quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique `fail-closed` quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

`STATUS = FAIL`

Se houver requisito arquitetural não especificado:

`STATUS = ARCHITECTURAL_BLOCKER`

Somente produza:

`STATUS = PASS`

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com `STATUS = PASS`;
- nenhuma `ARCHITECTURAL_BLOCKER` pendente;
- evidências registradas;
- commit registrado;
- `PHASE_XX_APPROVED.md` criado.

FASE 09 — AI Gateway

Objetivo

Criar a camada de integração com providers de LLM sem permitir autoridade operacional direta.

Escopo

Provider Adapter, modelos, credentials, timeout, retry, structured output, schema validation, prompt versioning, token accounting, mock provider e AI Run.

Fora do Escopo

Decision Pipeline completo e execução.

Dependências

Fases 01-08.

Visão Arquitetural

O AI Gateway é a única fronteira de integração com o LLM.

Nenhum provider recebe acesso ao broker, banco financeiro ou kill switch.

Critérios de Aceite

- Provider pode ser substituído por adapter.
- Output inválido é rejeitado.
- Timeout é tratado.
- Retry possui limites.
- Secrets não chegam ao frontend nem ao prompt.
- AI Run é auditável.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 09 – AI GATEWAY

=====

OBJETIVO:

Criar a camada de integração com providers de LLM sem permitir autoridade operacional direta.

ESCOPO:

Provider Adapter, modelos, credentials, timeout, retry, structured output, schema validation, prompt versioning, token accounting, mock provider e AI Run.

FORA DO ESCOPO:

Decision Pipeline completo e execução.

DEPENDÊNCIAS:

Fases 01-08.

VISÃO ARQUITETURAL:

O AI Gateway é a única fronteira de integração com o LLM.

Nenhum provider recebe acesso ao broker, banco financeiro ou kill switch.

CRITÉRIOS DE ACEITE:

- Provider pode ser substituído por adapter.
- Output inválido é rejeitado.
- Timeout é tratado.
- Retry possui limites.
- Secrets não chegam ao frontend nem ao prompt.
- AI Run é auditável.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 09 – AI GATEWAY

=====

OBJETIVO ESPERADO:

Criar a camada de integração com providers de LLM sem
permitir autoridade operacional direta.

ESCOPO ESPERADO:

Provider Adapter, modelos, credentials, timeout, retry,
structured output, schema validation, prompt versioning,
token accounting, mock provider e AI Run.

FORA DO ESCOPO:

Decision Pipeline completo e execução.

DEPENDÊNCIAS:

Fases 01-08.

VISÃO ARQUITETURAL:

O AI Gateway é a única fronteira de integração com o LLM.

Nenhum provider recebe acesso ao broker, banco financeiro
ou kill switch.

CRITÉRIOS DE ACEITE:

- Provider pode ser substituído por adapter.
- Output inválido é rejeitado.
- Timeout é tratado.
- Retry possui limites.
- Secrets não chegam ao frontend nem ao prompt.
- AI Run é auditável.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.

11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 10 — AI Agents & Decision Pipeline

Objetivo

Implementar Analyst, Critic e Decision Maker.

Escopo

MarketState -> Analyst -> Critic -> Decision Maker ->

TradeIntent, incluindo confidence, thesis, invalidation, entry, stop e target quando aplicável.

Fora do Escopo

Broker direto e bypass do Risk Engine.

Dependências

Fases 05, 08 e 09.

Visão Arquitetural

A IA produz intenção estruturada.

Ela nunca produz uma Order diretamente.

AI:

MarketState -> TradeIntent

Risk:

TradeIntent -> RiskDecision

Critérios de Aceite

- Analyst funciona.
- Critic funciona.
- Decision Maker produz schema válido.
- TradeIntent possui ai_run_id.
- Prompt version é registrada.
- Input/output hashes são registrados.
- Não existe caminho AI -> Broker.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====
FASE 10 – AI AGENTS & DECISION PIPELINE
=====

OBJETIVO:

Implementar Analyst, Critic e Decision Maker.

ESCOPO:

MarketState -> Analyst -> Critic -> Decision Maker ->
TradeIntent, incluindo confidence, thesis, invalidation,
entry, stop e target quando aplicável.

FORA DO ESCOPO:

Broker direto e bypass do Risk Engine.

DEPENDÊNCIAS:

Fases 05, 08 e 09.

VISÃO ARQUITETURAL:

A IA produz intenção estruturada.

Ela nunca produz uma Order diretamente.

AI:

MarketState -> TradeIntent

Risk:

TradeIntent -> RiskDecision

CRITÉRIOS DE ACEITE:

- Analyst funciona.
- Critic funciona.
- Decision Maker produz schema válido.
- TradeIntent possui ai_run_id.
- Prompt version é registrada.
- Input/output hashes são registrados.
- Não existe caminho AI -> Broker.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.

19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====
AUDITORIA DA FASE 10 – AI AGENTS & DECISION PIPELINE
=====

OBJETIVO ESPERADO:

Implementar Analyst, Critic e Decision Maker.

ESCOPO ESPERADO:

MarketState -> Analyst -> Critic -> Decision Maker ->
TradeIntent, incluindo confidence, thesis, invalidation,
entry, stop e target quando aplicável.

FORA DO ESCOPO:

Broker direto e bypass do Risk Engine.

DEPENDÊNCIAS:

Fases 05, 08 e 09.

VISÃO ARQUITETURAL:

A IA produz intenção estruturada.

Ela nunca produz uma Order diretamente.

AI:

MarketState -> TradeIntent

Risk:

TradeIntent -> RiskDecision

CRITÉRIOS DE ACEITE:

- Analyst funciona.
- Critic funciona.

- Decision Maker produz schema válido.
- TradeIntent possui ai_run_id.
- Prompt version é registrada.
- Input/output hashes são registrados.
- Não existe caminho AI -> Broker.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 11 — Decision + Risk + Execution Integration

Objetivo

Integrar o pipeline completo de decisão e execução simulada.

Escopo

Orchestration, correlation ID, idempotência, retries,

failure handling e fluxo completo:

MarketState -> AI -> TradeIntent -> Risk -> Execution ->

Paper Broker -> Fill -> Portfolio.

Fora do Escopo

Live trading.

Dependências

Fases 06-10.

Visão Arquitetural

Esta fase cria o coração operacional da V1.

Nenhuma camada pode saltar a Risk Engine.

Critérios de Aceite

- E2E de operação completa funciona.
- Cada etapa possui correlation_id.
- Falha em qualquer etapa produz estado seguro.
- Retry não duplica operação.
- AI failure bloqueia nova entrada.
- Proteção de posição não depende do LLM.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====
FASE 11 – DECISION + RISK + EXECUTION INTEGRATION
=====

OBJETIVO:

Integrar o pipeline completo de decisão e execução simulada.

ESCOPO:

Orchestration, correlation ID, idempotência, retries,
failure handling e fluxo completo:
MarketState -> AI -> TradeIntent -> Risk -> Execution ->
Paper Broker -> Fill -> Portfolio.

FORA DO ESCOPO:

Live trading.

DEPENDÊNCIAS:

Fases 06-10.

VISÃO ARQUITETURAL:

Esta fase cria o coração operacional da V1.

Nenhuma camada pode saltar a Risk Engine.

CRITÉRIOS DE ACEITE:

- E2E de operação completa funciona.
- Cada etapa possui correlation_id.
- Falha em qualquer etapa produz estado seguro.
- Retry não duplica operação.
- AI failure bloqueia nova entrada.
- Proteção de posição não depende do LLM.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:
 PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:
 resumo objetivo

IMPLEMENTED:
 o que foi encontrado implementado

MISSING:
 o que deveria existir e não existe

OUT_OF_SCOPE:
 funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:
 violações encontradas

TEST_RESULTS:
 testes executados e resultado

SECURITY_RESULTS:
 resultado

EVIDENCE:
 arquivos, testes e evidências

RECOMMENDATION:
 APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 11 – DECISION + RISK + EXECUTION INTEGRATION

=====

OBJETIVO ESPERADO:

Integrar o pipeline completo de decisão e execução simulada.

ESCOPO ESPERADO:

Orchestration, correlation ID, idempotência, retries,
 failure handling e fluxo completo:
 MarketState -> AI -> TradeIntent -> Risk -> Execution ->
 Paper Broker -> Fill -> Portfolio.

FORA DO ESCOPO:

Live trading.

DEPENDÊNCIAS:

Fases 06-10.

VISÃO ARQUITETURAL:

Esta fase cria o coração operacional da V1.

Nenhuma camada pode saltar a Risk Engine.

CRITÉRIOS DE ACEITE:

- E2E de operação completa funciona.
- Cada etapa possui correlation_id.
- Falha em qualquer etapa produz estado seguro.
- Retry não duplica operação.
- AI failure bloqueia nova entrada.
- Proteção de posição não depende do LLM.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 12 — Replay & Backtest

Objetivo

Criar o laboratório determinístico para replay e backtest.

Escopo

Dataset Registry, Experiment Registry, replay engine, backtest, seed, configuração versionada, fees, spread, slippage, métricas e prevenção de look-ahead.

Fora do Escopo

Produção/live trading.

Dependências

Fases 04-11.

Visão Arquitetural

Replay utiliza o mesmo pipeline de decisão sempre que possível, sem criar uma segunda implementação da estratégia.

Cada experimento identifica exatamente:

dataset

model

prompt

configuration

seed

Critérios de Aceite

- Dataset possui checksum.
- Experiment possui identificação completa.
- Replay é reproduzível.
- Mesmo input + seed produz mesmo resultado.
- Look-ahead é impossível.
- Fees/slippage podem ser simulados.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====
FASE 12 – REPLAY & BACKTEST
=====

OBJETIVO:

Criar o laboratório determinístico para replay e backtest.

ESCOPO:

Dataset Registry, Experiment Registry, replay engine, backtest, seed, configuração versionada, fees, spread, slippage, métricas e prevenção de look-ahead.

FORA DO ESCOPO:

Produção/live trading.

DEPENDÊNCIAS:

Fases 04-11.

VISÃO ARQUITETURAL:

Replay utiliza o mesmo pipeline de decisão sempre que possível, sem criar uma segunda implementação da estratégia.

Cada experimento identifica exatamente:

- dataset
- model
- prompt
- configuration
- seed

CRITÉRIOS DE ACEITE:

- Dataset possui checksum.
- Experiment possui identificação completa.
- Replay é reproduzível.
- Mesmo input + seed produz mesmo resultado.
- Look-ahead é impossível.
- Fees/slippage podem ser simulados.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.

20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 12 – REPLAY & BACKTEST

=====

OBJETIVO ESPERADO:

Criar o laboratório determinístico para replay e backtest.

ESCOPO ESPERADO:

Dataset Registry, Experiment Registry, replay engine, backtest, seed, configuração versionada, fees, spread, slippage, métricas e prevenção de look-ahead.

FORA DO ESCOPO:

Produção/live trading.

DEPENDÊNCIAS:

Fases 04-11.

VISÃO ARQUITETURAL:

Replay utiliza o mesmo pipeline de decisão sempre que possível, sem criar uma segunda implementação da estratégia.

Cada experimento identifica exatamente:

dataset
model
prompt
configuration
seed

CRITÉRIOS DE ACEITE:

- Dataset possui checksum.
- Experiment possui identificação completa.
- Replay é reproduzível.

- Mesmo input + seed produz mesmo resultado.
- Look-ahead é impossível.
- Fees/slippage podem ser simulados.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 13 — Frontend & Operational Dashboard

Objetivo

Criar a interface operacional completa.

Escopo

Dashboard, posições, ordens, portfolio, performance,

AI decisions, risk, market, configuration, providers,

LLM, prompts, audit, backtest e system status.

Fora do Escopo

Bypass de backend, Risk Engine ou broker.

Dependências

Fases 02-12.

Visão Arquitetural

Frontend é cliente.

Toda regra de segurança permanece no backend.

WebSocket fornece estado realtime, enquanto REST fornece operações e consultas adequadas.

Critérios de Aceite

- Posições abertas aparecem corretamente.
- P&L é consistente com backend.
- Ordens são exibidas.
- Risk status é visível.
- Kill switch é acessível segundo autorização.
- Secrets nunca aparecem.
- Frontend não consegue enviar Order diretamente ao broker.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 13 – FRONTEND & OPERATIONAL DASHBOARD

=====

OBJETIVO:

Criar a interface operacional completa.

ESCOPO:

Dashboard, posições, ordens, portfolio, performance, AI decisions, risk, market, configuration, providers, LLM, prompts, audit, backtest e system status.

FORA DO ESCOPO:

Bypass de backend, Risk Engine ou broker.

DEPENDÊNCIAS:

Fases 02-12.

VISÃO ARQUITETURAL:

Frontend é cliente.

Toda regra de segurança permanece no backend.

WebSocket fornece estado realtime, enquanto REST fornece operações e consultas adequadas.

CRITÉRIOS DE ACEITE:

- Posições abertas aparecem corretamente.
- P&L é consistente com backend.
- Ordens são exibidas.
- Risk status é visível.
- Kill switch é acessível segundo autorização.
- Secrets nunca aparecem.
- Frontend não consegue enviar Order diretamente ao broker.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 13 – FRONTEND & OPERATIONAL DASHBOARD

=====

OBJETIVO ESPERADO:

Criar a interface operacional completa.

ESCOPO ESPERADO:

Dashboard, posições, ordens, portfolio, performance,
AI decisions, risk, market, configuration, providers,
LLM, prompts, audit, backtest e system status.

FORA DO ESCOPO:

Bypass de backend, Risk Engine ou broker.

DEPENDÊNCIAS:

Fases 02-12.

VISÃO ARQUITETURAL:

Frontend é cliente.

Toda regra de segurança permanece no backend.

WebSocket fornece estado realtime, enquanto REST fornece
operações e consultas adequadas.

CRITÉRIOS DE ACEITE:

- Posições abertas aparecem corretamente.
- P&L é consistente com backend.
- Ordens são exibidas.
- Risk status é visível.
- Kill switch é acessível segundo autorização.
- Secrets nunca aparecem.
- Frontend não consegue enviar Order diretamente ao broker.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.

5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique `correlation_id` quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

```
STATUS = FAIL
```

Se houver requisito arquitetural não especificado:

```
STATUS = ARCHITECTURAL_BLOCKER
```

Somente produza:

```
STATUS = PASS
```

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 14 — Audit & Observability

Objetivo

Consolidar rastreabilidade completa e métricas.

Escopo

Audit Trail, structured logs, correlation chains, system metrics, AI metrics e trading metrics.

Fora do Escopo

Alteração de regras de negócio.

Dependências

Fases 01-13.

Visão Arquitetural

Toda operação deve ser reconstruível através da cadeia:

MarketState

->

AI Run

->

TradeIntent

->

RiskDecision

->

Order

->

Fill

->

Position

->

Portfolio

Critérios de Aceite

- Toda operação possui correlation_id.
- Audit events são persistidos.

- Logs são estruturados.
- Métricas técnicas existem.
- Métricas de IA existem.
- Métricas de trading existem.
- Uma operação pode ser reconstruída integralmente.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

```
=====
FASE 14 — AUDIT & OBSERVABILITY
=====
```

OBJETIVO:

Consolidar rastreabilidade completa e métricas.

ESCOPO:

Audit Trail, structured logs, correlation chains, system metrics, AI metrics e trading metrics.

FORA DO ESCOPO:

Alteração de regras de negócio.

DEPENDÊNCIAS:

Fases 01-13.

VISÃO ARQUITETURAL:

Toda operação deve ser reconstruível através da cadeia:

MarketState

->

AI Run

->

TradeIntent

->

RiskDecision

->

Order

->

Fill

->

Position

->

Portfolio

CRITÉRIOS DE ACEITE:

- Toda operação possui correlation_id.
- Audit events são persistidos.
- Logs são estruturados.
- Métricas técnicas existem.
- Métricas de IA existem.
- Métricas de trading existem.
- Uma operação pode ser reconstruída integralmente.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 14 – AUDIT & OBSERVABILITY

=====

OBJETIVO ESPERADO:

Consolidar rastreabilidade completa e métricas.

ESCOPO ESPERADO:

Audit Trail, structured logs, correlation chains, system metrics, AI metrics e trading metrics.

FORA DO ESCOPO:

Alteração de regras de negócio.

DEPENDÊNCIAS:

Fases 01-13.

VISÃO ARQUITETURAL:

Toda operação deve ser reconstruível através da cadeia:

MarketState

->

AI Run

->

TradeIntent

->

RiskDecision

->

Order

->

Fill

->

Position

->

Portfolio

CRITÉRIOS DE ACEITE:

- Toda operação possui correlation_id.
- Audit events são persistidos.
- Logs são estruturados.
- Métricas técnicas existem.
- Métricas de IA existem.
- Métricas de trading existem.
- Uma operação pode ser reconstruída integralmente.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;

- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 15 — Resilience, Security & Chaos

Objetivo

Validar que falhas não provocam comportamento inseguro.

Escopo

Chaos testing, restart testing, provider failures, database failures, Redis failure, worker failure, websocket failure, market gaps, duplicate events, duplicate orders, broker unavailable, security checks e backup/restore.

Fora do Escopo

Novas funcionalidades.

Dependências

Fases 01-14.

Visão Arquitetural

O sistema deve preferir PAUSE/SAFE STATE quando não consegue determinar corretamente seu estado.

Critérios de Aceite

- PostgreSQL down não gera novas ordens.
- Redis down não corrompe source of truth.
- Worker crash permite recovery.
- LLM timeout bloqueia nova entrada.
- Market gap bloqueia decisão quando necessário.
- Duplicate event não duplica operação.
- Restart durante execução é reconciliado.
- Backup/restore funciona.
- Secrets permanecem protegidos.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 15 – RESILIENCE, SECURITY & CHAOS

=====

OBJETIVO:

Validar que falhas não provocam comportamento inseguro.

ESCOPO:

Chaos testing, restart testing, provider failures, database failures, Redis failure, worker failure, websocket failure, market gaps, duplicate events, duplicate orders, broker unavailable, security checks e backup/restore.

FORA DO ESCOPO:

Novas funcionalidades.

DEPENDÊNCIAS:

Fases 01-14.

VISÃO ARQUITETURAL:

O sistema deve preferir PAUSE/SAFE STATE quando não consegue determinar corretamente seu estado.

CRITÉRIOS DE ACEITE:

- PostgreSQL down não gera novas ordens.
- Redis down não corrompe source of truth.
- Worker crash permite recovery.
- LLM timeout bloqueia nova entrada.
- Market gap bloqueia decisão quando necessário.
- Duplicate event não duplica operação.
- Restart durante execução é reconciliado.
- Backup/restore funciona.
- Secrets permanecem protegidos.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 15 – RESILIENCE, SECURITY & CHAOS

=====

OBJETIVO ESPERADO:

Validar que falhas não provocam comportamento inseguro.

ESCOPO ESPERADO:

Chaos testing, restart testing, provider failures, database failures, Redis failure, worker failure, websocket failure, market gaps, duplicate events, duplicate orders, broker unavailable, security checks e backup/restore.

FORA DO ESCOPO:

Novas funcionalidades.

DEPENDÊNCIAS:

Fases 01-14.

VISÃO ARQUITETURAL:

O sistema deve preferir PAUSE/SAFE STATE quando não consegue determinar corretamente seu estado.

CRITÉRIOS DE ACEITE:

- PostgreSQL down não gera novas ordens.
- Redis down não corrompe source of truth.
- Worker crash permite recovery.
- LLM timeout bloqueia nova entrada.
- Market gap bloqueia decisão quando necessário.
- Duplicate event não duplica operação.
- Restart durante execução é reconciliado.
- Backup/restore funciona.
- Secrets permanecem protegidos.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.

4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

```
STATUS = FAIL
```

Se houver requisito arquitetural não especificado:

```
STATUS = ARCHITECTURAL_BLOCKER
```

Somente produza:

```
STATUS = PASS
```

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

FASE 16 — Final V1 Certification

Objetivo

Certificar a V1 como PAPER_READY.

Escopo

Unit tests, integration tests, E2E, replay, backtest, security, chaos, recovery, backup/restore e paper trading.

Fora do Escopo

Live trading.

Dependências

Fases 01-15 aprovadas.

Visão Arquitetural

Esta fase não deve introduzir arquitetura nova.

Ela somente verifica se o sistema implementado corresponde ao Blueprint V1.1.

Critérios de Aceite

- Todas as fases anteriores aprovadas.
- Testes completos passam.
- Chaos testing passa.
- Security checks passam.
- Recovery passa.
- Backup/restore passa.
- Paper trading funciona.
- Audit trail é íntegro.
- Nenhum LiveBroker existe.
- V1_STATUS = PAPER_READY.

Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 16 – FINAL V1 CERTIFICATION

=====

OBJETIVO:

Certificar a V1 como PAPER_READY.

ESCOPO:

Unit tests, integration tests, E2E, replay, backtest,
security, chaos, recovery, backup/restore e paper trading.

FORA DO ESCOPO:

Live trading.

DEPENDÊNCIAS:

Fases 01-15 aprovadas.

VISÃO ARQUITETURAL:

Esta fase não deve introduzir arquitetura nova.

Ela somente verifica se o sistema implementado corresponde ao Blueprint V1.1.

CRITÉRIOS DE ACEITE:

- Todas as fases anteriores aprovadas.
- Testes completos passam.
- Chaos testing passa.
- Security checks passam.
- Recovery passa.
- Backup/restore passa.
- Paper trading funciona.
- Audit trail é íntegro.
- Nenhum LiveBroker existe.
- V1_STATUS = PAPER_READY.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT_OF_SCOPE:

funcionalidades indevidas

ARCHITECTURAL_VIOLATIONS:

violações encontradas

TEST_RESULTS:

testes executados e resultado

SECURITY_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX_REQUIRED / ARCHITECTURAL_REVIEW

=====

AUDITORIA DA FASE 16 – FINAL V1 CERTIFICATION

=====

OBJETIVO ESPERADO:

Certificar a V1 como PAPER_READY.

ESCOPO ESPERADO:

Unit tests, integration tests, E2E, replay, backtest,
security, chaos, recovery, backup/restore e paper trading.

FORA DO ESCOPO:

Live trading.

DEPENDÊNCIAS:

Fases 01-15 aprovadas.

VISÃO ARQUITETURAL:

Esta fase não deve introduzir arquitetura nova.

Ela somente verifica se o sistema implementado corresponde
ao Blueprint V1.1.

CRITÉRIOS DE ACEITE:

- Todas as fases anteriores aprovadas.
- Testes completos passam.
- Chaos testing passa.
- Security checks passam.
- Recovery passa.
- Backup/restore passa.
- Paper trading funciona.
- Audit trail é íntegro.
- Nenhum LiveBroker existe.
- V1_STATUS = PAPER_READY.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.

3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique `correlation_id` quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

```
STATUS = FAIL
```

Se houver requisito arquitetural não especificado:

```
STATUS = ARCHITECTURAL_BLOCKER
```

Somente produza:

```
STATUS = PASS
```

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

Gate da Fase

A fase somente pode ser considerada aprovada quando:

- implementação concluída;
- testes passando;
- critérios de aceite demonstrados;
- auditoria com STATUS = PASS;
- nenhuma ARCHITECTURAL_BLOCKER pendente;
- evidências registradas;
- commit registrado;
- PHASE_XX_APPROVED.md criado.

18. FINAL V1 CERTIFICATION

A certificação final não introduz nova arquitetura.

Ela verifica se o sistema implementado corresponde ao Blueprint V1.1.

```
All Phases PASS
|
v
Full E2E Test
|
v
Replay / Backtest
|
v
Security Audit
|
v
Chaos Testing
|
v
Recovery Test
|
v
Backup / Restore
|
v
Paper Trading
|
v
FINAL APPROVAL
|
v
V1_STATUS = PAPER_READY
```

19. Architecture Freeze

O Blueprint V1.1 é a autoridade arquitetural do projeto.

Nenhum agente, desenvolvedor ou processo de implementação pode modificar arquitetura, contratos, máquinas de estado, limites de risco ou escopo sem uma alteração formal de versão do Blueprint e respectivo ADR.

Qualquer requisito não especificado deve ser tratado como:

ARCHITECTURAL_BLOCKER

e não como autorização para o agente tomar uma decisão própria.

20. V1 Definition

A V1 será considerada concluída quando o sistema estiver:

V1_STATUS = PAPER_READY

A V1 NÃO será considerada:

LIVE_READY

Live Broker não faz parte da V1.

APPENDIX B — V1.2 SOURCE DIGEST

The V1.2 source is retained as a traceability source, but repeated evidence blocks are intentionally NOT copied into the main blueprint.

Source SHA-256:

335408780f4807c9e5ee59c2070d6110c7c3e375452d1c038f67afcb7e9fdd42

V1.2 contributions incorporated into V1.3 include:

- Replay-first validation;
- frozen technology stack;
- measurable acceptance criteria;
- explicit phase audits;
- PASS_WITH_WARNINGS;
- release-gate thinking;
- LLM governance;
- 16-phase implementation structure;
- academic scope removal.

The full V1.2 source remains external traceability material; V1.3 itself is the implementation authority.